

Obs. A modularização do projeto ficou ruim.. Inclui dentro de basic? Ainda, todas as implementações dentro de kernel.c? Melhor acertar isso.

PROJETO 2: PRIMEIROS PASSOS PARA O PROJETO DE UM SO

Gustavo Borges Peres da Silva¹, Michael P. C. Silva¹, Rafael S. Melo¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

{gustavo.silva, michael.silva}@discente.ufcat.edu.br,
rafinhalelograma@hotmail.com

Objetivos

- Primeiros passos para o desenvolvimento de um SO e testes em um ambiente de máquina virtual;
- Geração de kernels executáveis em formatos “.bin” e como ISO;
- Compreensão básica da estrutura de boot;
- Operações de I/O em tela em baixo nível.

1. Tarefa 1: preparação do sistema e criação do seu primeiro kernel

1.1. Máquina utilizada

Para este projeto, será utilizado a mesma máquina hospedeira Windows 11 apresentada no tópico “1.1 Descrição” do projeto 1, cujo a mesma possui também um guest/convidado com a distro Debian Linux devido sua disponibilidade e licenciamento deste sistema que será utilizada para o andamento da tarefa 1, responsável por preparar o sistema e criar o primeiro kernel. Vale lembrar também, que nessa máquina possui dois usuários (so2 e vboxuser), portanto, o usuário que iremos utilizar é o vboxuser pois possui privilégios administrativos necessários para darmos continuidade.

1.2. Instalação de pacotes

O primeiro passo que iremos realizar nessa prática de desenvolvimento de SO é a preparação do nosso SO Linux. Sendo assim, iremos criar um shell script para o ajuste/instalação de alguns pacotes necessários. Na tarefa não foi especificado o local de criação das pastas raízes, sendo assim, optamos por criar uma pasta chamada Projeto_2 no diretório: home/vboxuser/, pois isso irá nos auxiliar na navegação e deixar a estrutura do nosso projeto mais organizada. Abaixo, temos a Figura 1, para que possamos nos orientar sobre a arquitetura de organização do linux possibilitando navegar sobre os diretórios do sistema usando o comando “cd” e “ls” sem quaisquer problemas e também mostrando como foi criado a pasta utilizando-se o comando mkdir.

```
vboxuser@Debian12: ~  
vboxuser@Debian12:/$ ls  
bin    home    lib32    media    root    sys    vmlinuz  
boot   initrd.img  lib64    mnt      run     tmp    vmlinuz.old  
dev     initrd.img.old  libx32   opt      sbin    usr  
etc      lib      lost+found  proc     srv     var  
vboxuser@Debian12:/$ cd home  
vboxuser@Debian12:/home$ ls  
ggust   vboxuser  
vboxuser@Debian12:/home$ cd vboxuser  
vboxuser@Debian12:~$ ls  
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  
vboxuser@Debian12:~$ mkdir Projeto_2  
vboxuser@Debian12:~$ ls  
Desktop  Downloads  Pictures  Public  Videos  
Documents  Music      Projeto_2  Templates  
vboxuser@Debian12:~$
```

Figura 1: definição e criação da pasta raiz do projeto 2.

Agora iremos criar um arquivo setup.sh porém para isso, iremos utilizar o editor de texto chamado nano disponível no linux. Portanto, para isso, entre no diretório e escreva nano para entrar no editor de texto.

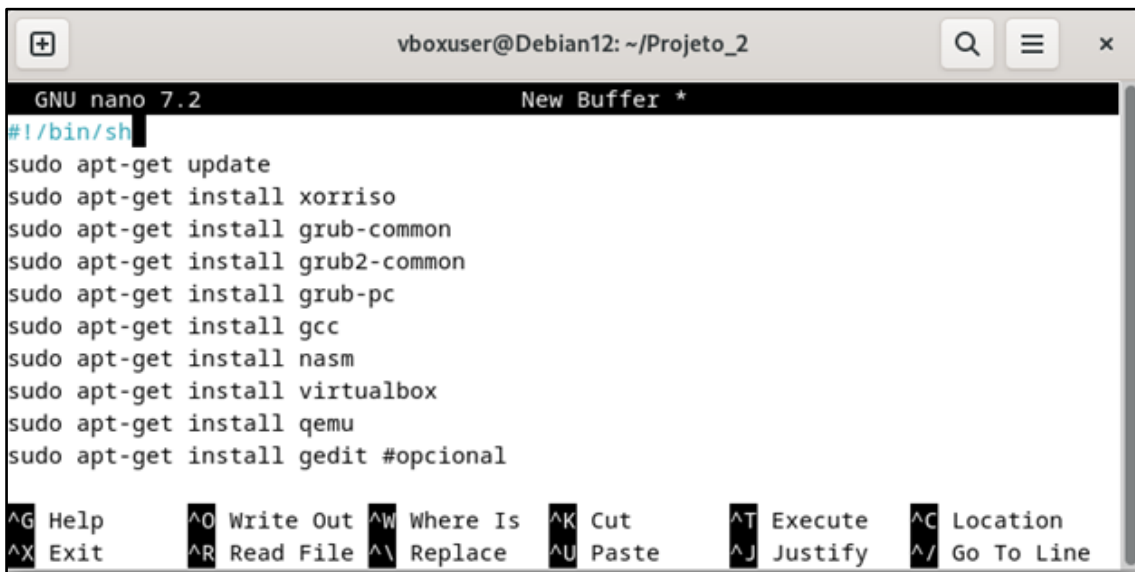
```
vboxuser@Debian12: ~/Projeto_2  
vboxuser@Debian12:~$ cd Projeto_2/  
vboxuser@Debian12:~/Projeto_2$ nano
```

Figura 2: nano editor.

Agora iremos colar o código fornecido pelo professor (Código 1), que se trata de várias linhas de comandos que irão realizar o ajuste ou instalação de alguns pacotes necessários para esta prática de laboratório.

```
#!/bin/sh  
sudo apt-get update  
sudo apt-get install xorriso  
sudo apt-get install grub-common  
sudo apt-get install grub2-common  
sudo apt-get install grub-pc  
sudo apt-get install gcc  
sudo apt-get install nasm  
sudo apt-get install virtualbox  
sudo apt-get install qemu  
sudo apt-get install gedit #opcional
```

Código 1 – Conteúdo do arquivo script setup.sh.



```
GNU nano 7.2 New Buffer *
#!/bin/sh
sudo apt-get update
sudo apt-get install xorriso
sudo apt-get install grub-common
sudo apt-get install grub2-common
sudo apt-get install grub-pc
sudo apt-get install gcc
sudo apt-get install nasm
sudo apt-get install virtualbox
sudo apt-get install qemu
sudo apt-get install gedit #opcional
```

Help Write Out Where Is Cut Execute Location
Exit Read File Replace Paste Justify Go To Line

Figura 3 – Criação do arquivo setup.sh via nano.

Para salvar iremos utilizar o comando ctrl+x (Figura 4) dentro do editor nano, isso irá fazer o nano fechar, porém ele te avisa que o arquivo não foi salvo pois estamos criando um buffer novo, sendo assim, digite Y (Figura – 5) para salvar as modificações do buffer.



Figura 4 – Salvando o arquivo setup.sh via nano parte 1.

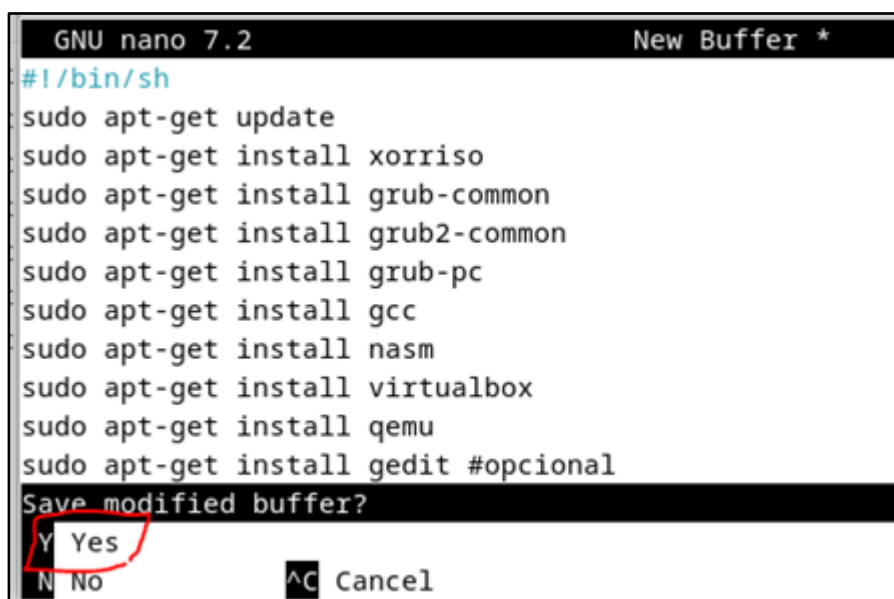
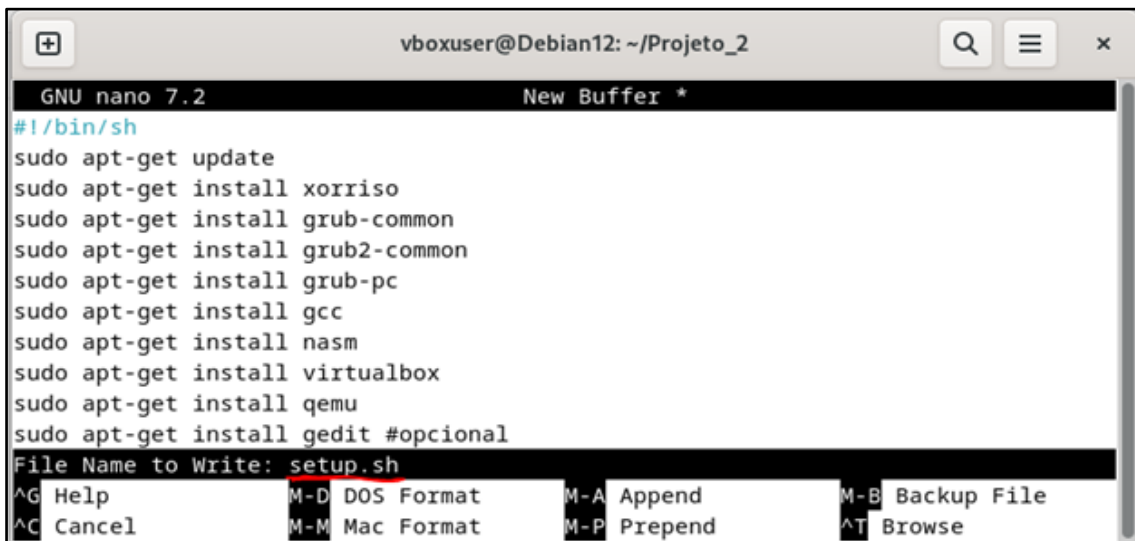


Figura 5 – Salvando o arquivo setup.sh via nano parte 2.

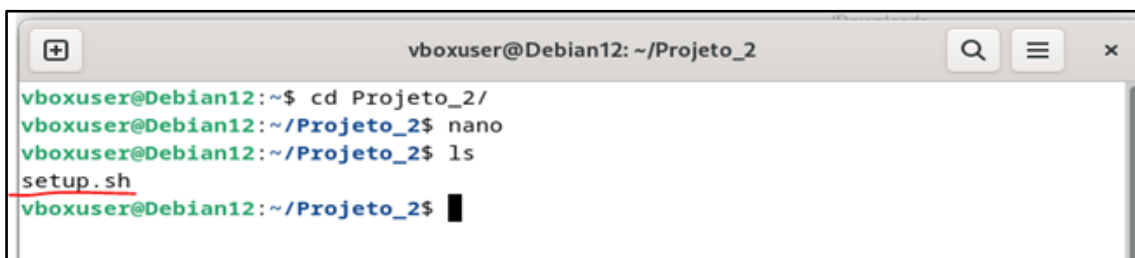
Agora será solicitado para que seja escrito o nome do arquivo, e conforme combinamos, iremos escrever setup.sh, após isso basta pressionar enter para concluir.



```
vboxuser@Debian12: ~/Projeto_2
GNU nano 7.2 New Buffer *
#!/bin/sh
sudo apt-get update
sudo apt-get install xorriso
sudo apt-get install grub-common
sudo apt-get install grub2-common
sudo apt-get install grub-pc
sudo apt-get install gcc
sudo apt-get install nasm
sudo apt-get install virtualbox
sudo apt-get install qemu
sudo apt-get install gedit #opcional
File Name to Write: setup.sh
^G Help      M-D DOS Format  M-A Append      M-B Backup File
^C Cancel    M-M Mac Format  M-P Prepend     ^T Browse
```

Figura 6 – Salvando o arquivo setup.sh via nano parte 3.

E por último verifique se o script foi criado corretamente usando o comando ls visto anteriormente.



```
vboxuser@Debian12: ~/Projeto_2
vboxuser@Debian12:~$ cd Projeto_2/
vboxuser@Debian12:~/Projeto_2$ nano
vboxuser@Debian12:~/Projeto_2$ ls
setup.sh
vboxuser@Debian12:~/Projeto_2$
```

Figura 7 – setup.sh criado.

Agora iremos verificar as propriedades de leitura/escrita/execução do nosso arquivo script para que o shell script possa ser executado, sendo assim, utilizaremos o comando `ls -l setup.sh` para exibir as informações detalhadas sobre o nosso arquivo.



```
vboxuser@Debian12: ~/Projeto_2
vboxuser@Debian12:~/Projeto_2$ ls -l setup.sh
-rw-r--r-- 1 vboxuser vboxuser 301 Dec  4 09:20 setup.sh
vboxuser@Debian12:~/Projeto_2$
```

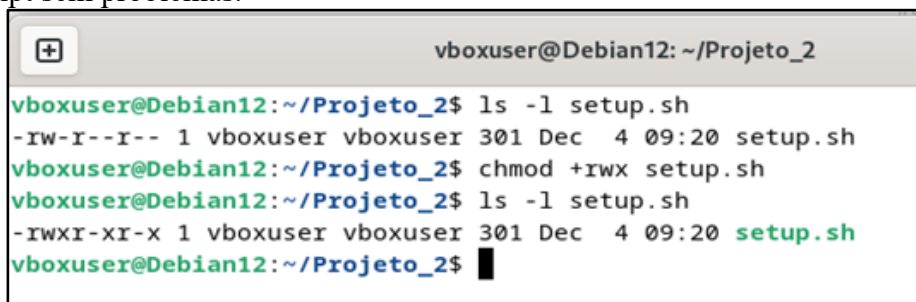
Figura 8 – Verificando permissões do setup.sh.

Neste caso, `-rw-r--r--` é uma sequência de caracteres que representa as permissões do arquivo para o proprietário, grupo e outros, respectivamente.

- O primeiro caractere (-) indica que é um arquivo regular.
- Os três caracteres seguintes (rw-) indicam que o proprietário (vboxuser) tem permissão de leitura e escrita, mas não de execução.
- Os três caracteres seguintes (r--) indicam que o grupo (também vboxuser) tem permissão apenas de leitura.

- Os últimos três caracteres (r--) indicam que outros usuários também têm permissão apenas de leitura.

Sendo assim, para atribuímos permissão de execução será utilizado o comando **chmod +rwx setup.sh** para o ajuste de maneira apropriada para que possamos executar o shell script sem problemas.



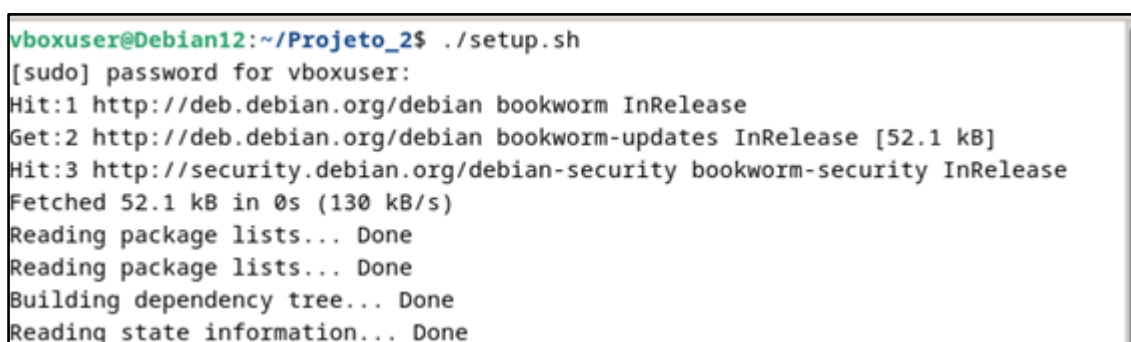
```
vboxuser@Debian12: ~/Projeto_2
vboxuser@Debian12:~/Projeto_2$ ls -l setup.sh
-rw-r--r-- 1 vboxuser vboxuser 301 Dec  4 09:20 setup.sh
vboxuser@Debian12:~/Projeto_2$ chmod +rwx setup.sh
vboxuser@Debian12:~/Projeto_2$ ls -l setup.sh
-rwxr-xr-x 1 vboxuser vboxuser 301 Dec  4 09:20 setup.sh
vboxuser@Debian12:~/Projeto_2$
```

Figura 9 – Atribuição de permissões para o setup.sh.

Agora o arquivo setup.sh tem permissões de leitura, escrita e execução para o usuário vboxuser. Isso é representado pela sequência de caracteres -rwxr-xr-x.

- O primeiro conjunto de caracteres (rwx) indica as permissões para o proprietário (vboxuser): leitura, escrita e execução.
- O segundo conjunto de caracteres (r-x) indica as permissões para o grupo (também vboxuser): leitura e execução.
- O terceiro conjunto de caracteres (r-x) indica as permissões para outros usuários: leitura e execução.

Agora com as permissões atribuídas, vamos executar o script (Figura 10).



```
vboxuser@Debian12:~/Projeto_2$ ./setup.sh
[sudo] password for vboxuser:
Hit:1 http://deb.debian.org/debian bookworm InRelease
Get:2 http://deb.debian.org/debian bookworm-updates InRelease [52.1 kB]
Hit:3 http://security.debian.org/debian-security bookworm-security InRelease
Fetched 52.1 kB in 0s (130 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Figura 10 – Executando o setup.sh.

Após instalar tudo, em nosso caso, tivemos dois erros. O primeiro foi para instalar o qemu pois devido a distro utilizada o comando para instalar não seria o mesmo fornecido pelo professor, sendo assim, tivemos de modificar 1 linha do script para conseguir instalar o qemu, a modificação foi: **sudo apt-get install qemu-system**. O segundo erro foi para instalar o virtualbox, pois no debian possui algumas limitações, sendo assim, tivemos que instalar usando o repositório oficial do virtualbox através da inclusão do repositório manualmente e chave GPG do Oracle para garantir a autenticidade dos pacotes.

Além disso, no script também será necessário realizar a modificação da linha de instalação do virtualbox, pois ao executar o script normalmente, o sistema irá informar

que existem algumas versões disponíveis, em nosso caso optamos por utilizar virtualbox-7.0. Após realizar tudo isso, basta executar o script novamente com as alterações realizadas (Código 2).

Segue abaixo os códigos utilizados para adicionar repositório e chave de autenticação do virtualbox:

```
#!/bin/sh
sudo sh -c 'echo "deb [arch=amd64] http://download.virtualbox.org/virtualbox/debian bookworm contrib" >
/etc/apt/sources.list.d/virtualbox.list'
wget -q https://www.virtualbox.org/download/oracle_vbox_2016.asc -O- | sudo apt-key add -
sudo apt-get update
sudo apt-get install xorriso
sudo apt-get install grub-common
sudo apt-get install grub2-common
sudo apt-get install grub-pc
sudo apt-get install gcc
sudo apt-get install nasm
sudo apt-get install virtualbox-7.0
sudo apt-get install qemu-system
sudo apt-get install gedit #opcional
```

Código 2 – Atualização do script setup.sh.

1.3. Preparação do projeto

A seguir iremos criar uma pasta chamada “so” e iremos prepará-la para o projeto.



```
vboxuser@Debian12:~/Projeto_2$ mkdir so
vboxuser@Debian12:~/Projeto_2$ ls
setup.sh  so
```

Figura 11 – Criar pasta “so”.

Nesta pasta serão criados vários arquivos utilizando-se os mesmos passos das figuras 2 a 7. Vale ressaltar que, ao decorrer do nosso relatório, iremos apresentar cada parte da criação dos arquivos, portanto, dentre os arquivos mencionados, temos:

- Criação do arquivo kernel.asm. Que será o ponto de entrada nosso (entry point) do kernel do SO que está sendo criado.
- Criação do arquivo kernel.c e link.ld. Cujo o arquivo kernel é destinado a ser executado no início do kernel do sistema operacional e o “link.ld” que servirá para realizar a união dos arquivos binários gerados pela compilação do do “kernel.c”.
- Criação de um shell script chamado build.sh utilizando-se os mesmos procedimentos das Figuras 8 e 9. Que será um script facilitador para as operações de compilações e combinações que iremos realizar.
- Criação de um arquivo grub.cfg. Que será o nosso gerenciador de inicialização do nosso sistema operacional.
- Criação de um arquivo simple.iso. Que será a nossa imagem gerada do nosso sistema operacional para testarmos posteriormente na máquina virtual.

Antes de continuarmos, no presente momento é interessante entendermos algumas contextualizações, sendo assim, o bootloader é um pequeno pedaço de código que fica no início do disco e tem a função de iniciar o sistema operacional. Ele aponta para um local específico onde o código do sistema operacional (SO) começa. Esse local específico é conhecido como o ponto de entrada do kernel.

O kernel é a parte principal do sistema operacional e é dividido em duas partes: a primeira parte é o ponto de entrada, que precisa estar em um local específico na memória. Esse ponto de entrada indica onde o código real do kernel começa. Então, simplifiadamente, temos uma sequência como esta: bootloader → kernel.asm → kernel.c.

Os arquivos mencionados, como kernel.asm e kernel.c, são simplesmente arquivos de texto contendo o código-fonte do kernel. Para transformá-los em algo que o computador pode executar, precisamos inserir código nesses arquivos e depois compilar usando as ferramentas apropriadas. A compilação é o processo de transformar o código-fonte em código executável que o computador pode entender e executar. Agora que resumimos, contextualizamos, vamos apresentar os arquivos que iremos criar e explicá-los o que cada um faz, de início, iremos iniciar primeiro com a criação do arquivo kernel.asm conforme o Código 3.

```
bits 32
section .text
    align 4
    dd 0x1BADB002
    dd 0x00
    dd - (0x1BADB002+0x00)

global start
extern kmain
start:
    cli
    call kmain
    hlt
```

Código 3 – Conteúdo do arquivo kernel.asm.

Além disso, iremos criar outros dois arquivos, o kernel.c e link.ld conforme os codigos abaixo (Codigo 4 e 5).

```
kmain()
{
    char *vidmem = (char*) 0xb8000;
    vidmem[0] = 'X';
    vidmem[1] = 0x02; /* cor do caractere: verde */
}
```

Código 4 – Conteúdo do arquivo kernel.c.

```
OUTPUT_FORMAT(elf32-i386)
ENTRY(start)
SECTIONS
{
```



```
. = 0x100000;
.text : { *(.text) }
.data : { *(.data) }
.bss : { *(.bss) }
}
```

Código 5 – Conteúdo do arquivo link.ld.

```
vboxuser@Debian12:~/Projeto_2$ cd so/
vboxuser@Debian12:~/Projeto_2/so$ nano kernel.asm
vboxuser@Debian12:~/Projeto_2/so$ nano kernel.c
vboxuser@Debian12:~/Projeto_2/so$ nano link.ld
```

Figura 12 – Visão geral da criação dos arquivos mostrados.

No código 3 (conteúdo kernel.asm), é importante saber que:

- O bits 32 não é uma instrução em Assembly. É uma diretiva para o NASM assembler que especifica que este deve gerar código para ser executado em um processador em modo de 32 bits. Não é obrigatório no exemplo, mas é incluído como parte da boa prática de ser explícito;
- A **segunda linha** (section .text) inicia a seção de texto (a.k.a seção de código). Aqui é onde se coloca todo o código. Estes são definidos usando-se a diretiva dd;
- A partir da **terceira linha**, são inseridas especificações para que se possa utilizar o GRUB (Grand Unified Bootloader), por isso o kernel é criado com suporte a multiboot. De acordo com a especificação, o kernel deve conter um cabeçalho nos primeiros 8 KB. Além disso, o cabeçalho deverá possuir três campos que estão alinhados em 4 bytes (align 4);
- 0x1BADB002 é o chamado “número mágico” (**magic number**). Este é o número que o bootloader deve encontrar para se assegurar que este é um SO que aceita multi-boot.
- O próximo campo é destinado a **flags** e será ignorado (dd 0x00);
- O último campo é o **checksum**. Este, ao ser somado ao campo de magic number e flags, deve dar 0.
- Será criada uma função chamada start. Esta função é definida como global para que possa ser acessada usando-se o **linker**. A execução desta função será iniciada com o linker.
- Por último é criada uma nova função externa chamada **kmain**, a qual estará em kernel.c.
- Na função **start**, a primeira linha **cli**, limpa as interrupções. Neste contexto, as interrupções são alguns serviços oferecidos pela BIOS, os quais realizam algumas operações básicas, como mapeamento para a tela, rastreamento de teclado etc. Ao passar para o modo de 32 bits, ou modo protegido, não é mais possível usar estes serviços. Assim, é necessário limpar as interrupções, já que estas não serão usadas novamente, e assim, liberar espaço para o código C que será carregado em seguida.
- A linha **call kmain** manda o processador continuar sua execução na função kmain, a qual será definida em kernel.c.
- A última linha **hlt**, “pausa” a execução da CPU considerando o endereço corrente.

Portanto, esse trecho de código em Assembly serve para **chamar uma função externa** escrita em C e, em seguida, **interromper o fluxo** do programa. Para garantir que isso funcione conforme o esperado, será utilizado um **script de ligação** posteriormente. Esse **script vincula** os arquivos objeto para criar o executável final do kernel. No script de ligação (código 5), especificaremos explicitamente que queremos que o arquivo binário seja carregado no endereço 0x100000, onde esperamos que o kernel seja armazenado. O bootloader será responsável por iniciar o ponto de entrada do kernel.

Agora, só precisamos implementar a função kmain no arquivo kernel.c (Código 4). Essa função irá imprimir um caractere na tela do computador ('X' na cor verde) para demonstrar que nosso código está funcionando. Além disso, estamos focando na compilação e na geração dos arquivos necessários para um boot real do nosso futuro sistema operacional (muito simples). Neste momento, podemos considerar que temos um sistema operacional funcional.

E agora iremos criar um arquivo shell script chamado build.sh (código 6) e atribuir permissões de leitura, gravação e execução utilizando os mesmos passos realizados anteriormente (Figura 13).

```
#!/bin/sh
nasm -f elf32 kernel.asm -o kasm.o
gcc -m32 -c -w kernel.c -o kc.o
ld -m elf_i386 -T link.ld -o kernel.bin kasm.o kc.o
```

Código 6 – Conteúdo do arquivo build.sh.

```
vboxuser@Debian12:~/Projeto_2/so$ nano build.sh
vboxuser@Debian12:~/Projeto_2/so$ ls
build.sh kernel.asm kernel.c link.ld
vboxuser@Debian12:~/Projeto_2/so$ ls -l build.sh
-rw-r--r-- 1 vboxuser vboxuser 129 Dec  7 02:08 build.sh
vboxuser@Debian12:~/Projeto_2/so$ chmod +rwx build.sh
vboxuser@Debian12:~/Projeto_2/so$ ls -l build.sh
-rwxr-xr-x 1 vboxuser vboxuser 129 Dec  7 02:08 build.sh
vboxuser@Debian12:~/Projeto_2/so$
```

Figura 13 – Visão geral da configuração do script build.sh.

Por outro lado, para conseguirmos compilar este código, será necessário compilar inicialmente o código kernel.asm, o qual representa o ponto de entrada de nosso kernel. Na sequência, também precisamos compilar o arquivo kernel.c, o qual será indicado para execução a partir do código kernel.asm. Após se compilar estes dois arquivos, iremos combiná-los, com o intuito de se construir um binário final para execução.

Portanto, para iniciarmos a compilação destes códigos, iremos executar o script apresentado no código 6. Sendo assim, abra o terminal no diretório correspondente e execute o script conforme Figura 14.

```
vboxuser@Debian12:~/Projeto_2/so$ ./build.sh
ld: warning: kasm.o: missing .note.GNU-stack section implies executable stack
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
ld: warning: kernel.bin has a LOAD segment with RWX permissions
vboxuser@Debian12:~/Projeto_2/so$ █
```

Figura 14 – build.sh executado com a presença de anotações e avisos.

Como podemos observar, tivemos 2 avisos e uma nota, sendo assim explicaremos abaixo cada uma delas:

- **“ld: warning: kasm.o: missing .note.GNU-stack section implies executable stack”**: este aviso indica que o objeto kasm.o do nosso código em assembly não possui uma seção chamada .note.GNU-stack. Essa seção geralmente é usada para indicar ao sistema operacional se a pilha de execução é executável ou não. Apesar de ser um aviso, isso geralmente não é um problema significativo porém de acordo com o blog da Red Hat, o Nick Clifton [2] disse que se trata de avisos para alertar desenvolvedores quando as condições de segurança não estão em vigor, essas condições seriam no caso de certificar-se de que nem a pilha nem o segmento de dados possam conter o código e que o segmento de código não possa ser modificado.
- **“ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker”**: Esta nota indica que o comportamento anterior (referente à pilha executável mencionada no aviso anterior) está obsoleto e será removido em versões futuras do linker (ld). Isso significa que futuras versões do linker podem não emitir mais esse aviso [4].
- **ld: warning: kernel.bin has a LOAD segment with RWX permissions**: quando carregado na memória, um programa é normalmente dividido em diferentes segmentos. Um para código, um para dados e talvez alguns outros para vários usos especiais. Os segmentos têm atributos como leitura, gravação e executável e se eles têm todos os três, então eles são vulneráveis ao ataque. Então, nesses casos, o linker irá produzir esse aviso [2].

Conquanto, após rodarmos o script, foi criado mais outros 3 arquivos resultantes dessa compilação (Figura 15). Além disso, sobre os avisos deu a entender que basicamente foi criado um arquivo kernel.bin cujo o mesmo possui permissões rwx (leitura, gravação e execução) e de acordo com o aviso isso significa que são arquivos vulneráveis a ataques porém não precisamos nos preocupar com isto, iremos prosseguir normalmente com o procedimento.

```
vboxuser@Debian12:~/Projeto_2/so$ ls
build.sh  kasm.o  kc.o  kernel.asm  kernel.bin  kernel.c  link.ld
vboxuser@Debian12:~/Projeto_2/so$ ls -l kernel.bin
-rwxr-xr-x 1 vboxuser vboxuser 4936 Dec  7 02:56 kernel.bin
vboxuser@Debian12:~/Projeto_2/so$ █
```

Figura 15 – Verificando as permissões de kernel.bin.

Basicamente, o script (Código 6) fez com que o código Assembly no arquivo `kernel.asm` fosse compilado usando o Netwide Assembler (`nasm`) para gerar um arquivo no formato Executable and Linking Format (ELF) de 32 bits. O ELF é um formato flexível e multiplataforma que não está vinculado a uma arquitetura de CPU específica.

O arquivo C, `kernel.c`, é compilado com o GCC usando a opção `-m32` para gerar código de 32 bits. Após a compilação, dois arquivos binários resultantes são produzidos: `kasm.o` e `kc.o`. Para unir esses arquivos, a ordem é crucial, e o ponto de entrada (entry point) deve ser inserido primeiro, ou seja, `kasm.o`.

E para unir os arquivos binários gerados, usamos o linker, especificando o arquivo `link.ld` (Código 5). O formato de saída é ELF de 32 bits, com a arquitetura explicitamente definida como `i386`. O ponto de entrada (ENTRY) é a função `start` de `kernel.asm`. O kernel inicia sua execução a partir dessa função, comunica com `kmain` em `kernel.c`, e em seguida, pausa sua execução usando a instrução `hlt` em `kernel.asm`. Após a união, o resultado é o arquivo executável `kernel.bin`, que contém o código combinado de `kernel.asm` e `kernel.c`. Seções, como `.text` em `kernel.asm`, indicam onde o código será armazenado. Outras seções serão explicadas conforme necessário, sendo ignoradas por enquanto.

Agora, para testarmos o kernel gerado, iremos utilizar o software emulador `qemu` por recomendação do docente Dalton Matsuo Tavares.

Sendo assim, o comando que iremos executar é: `qemu-system-i386 --kernel kernel.bin`.

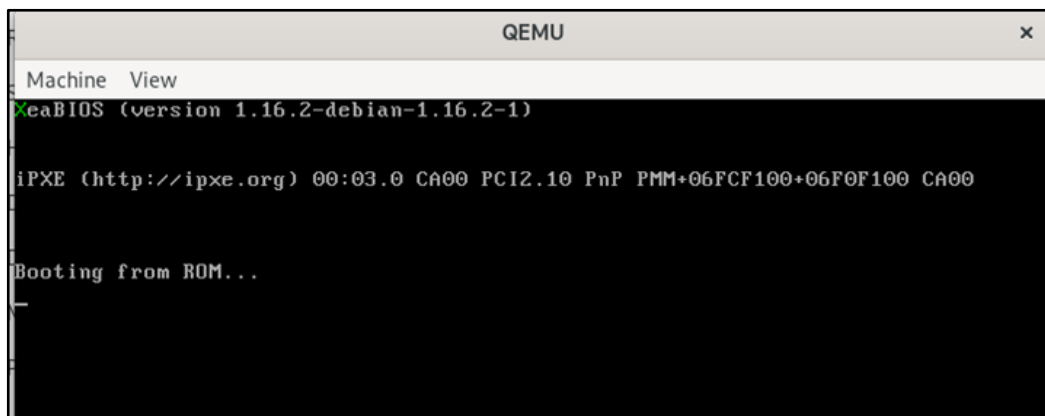


Figura 16 – Inicializando máquina virtual x86 (32 bits) usando o QEMU.

Feito isso, irá abrir a janela da nossa máquina, está funcionando normalmente devido o X de cor verde conforme configuramos no código 4. Além disso, infelizmente eu não consegui tirar o print no momento, mas aparece uma opção de `allow` ou `deny` relacionada ao `release grab action` no `debian`, porém eu não havia selecionado, sendo assim, na janela aparece uma mensagem de atalho caso queria fechar a janela ou mover/redimensionar utilizando o `Ctrl+Alt+G` para soltar o cursor do mouse pois ao clicar na janela estaremos com foco em outra máquina.

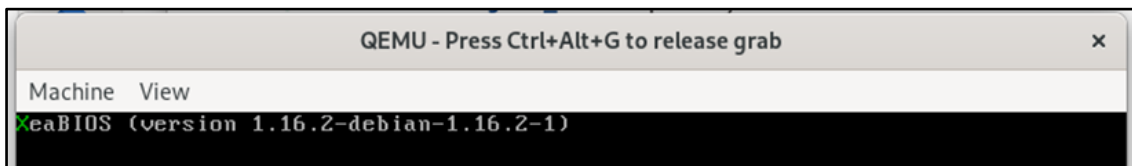


Figura 17 – Atalho do release grab dentro do QEMU.

Além disso, optamos por criar um novo script run.sh (Codigo 7) para execução automática do kernel conforme a Figura 18. Colocamos permissões rwx (leitura, gravação e execução).

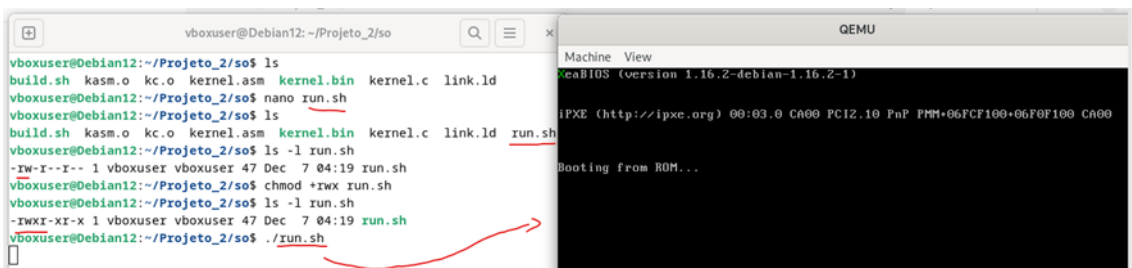


Figura 18 – Criação script run.sh, permissões e execução.

```
#!/bin/sh
qemu-system-i386 --kernel kernel.bin
```

Código 7 – Conteúdo do arquivo run.sh.

Por fim, para uma questão de completude, o próximo passo é a conversão do arquivo kernel.bin em um arquivo ISO, de modo a permitir a execução de nosso SO simples, por exemplo, no VirtualBox.

Para a geração do mesmo, criamos a estrutura de diretórios da seguinte forma: criamos um diretório chamado basic. Internamente ao diretório criado, criamos um novo diretório chamado boot (basic/boot) e por último, criamos um diretório chamado grub (basic/boot/grub). Dentro do diretório grub, criamos um arquivo chamado grub.cfg (Código 8).

```
vboxuser@Debian12:~/Projeto_2/so$ mkdir basic
vboxuser@Debian12:~/Projeto_2/so$ cd basic
vboxuser@Debian12:~/Projeto_2/so/basic$ mkdir boot
vboxuser@Debian12:~/Projeto_2/so/basic$ cd boot
vboxuser@Debian12:~/Projeto_2/so/basic/boot$ mkdir grub
vboxuser@Debian12:~/Projeto_2/so/basic/boot$ cd grub
vboxuser@Debian12:~/Projeto_2/so/basic/boot/grub$ nano grub.cfg
vboxuser@Debian12:~/Projeto_2/so/basic/boot/grub$ ls
grub.cfg
vboxuser@Debian12:~/Projeto_2/so/basic/boot/grub$ SS
```

Figura 19 – Criação da estrutura de diretórios.

```
set default = 0
```

```
set timeout = 0
menuentry "basicOS" {
    set root='(hd96)'
    multiboot /boot/kernel.bin
}
```

Código 8 – Conteúdo do arquivo grub.cfg.

Vale ressaltar que, para que esse arquivo de configuração funcione corretamente, deverá ser inserido o nosso arquivo kernel.bin gerado anteriormente, no diretório boot criado. Entretanto, iremos move-lo (Figura 20) e não copia-lo, sendo assim, o nosso script run.sh não irá funcionar corretamente pois não conseguirá encontrar a localização do arquivo kernel.bin, sendo assim iremos modificá-lo também com o path correto (Código 9).



```
vboxuser@Debian12:~/Projeto_2/so$ ls
basic      kasm.o    kernel.asm kernel.c  run.sh
build.sh   kc.o      kernel.bin link.ld
vboxuser@Debian12:~/Projeto_2/so$ mv kernel.bin ./basic/boot
vboxuser@Debian12:~/Projeto_2/so$ cd ./basic/boot
vboxuser@Debian12:~/Projeto_2/so/basic/boot$ ls
grub  kernel.bin
vboxuser@Debian12:~/Projeto_2/so/basic/boot$
```

Figura 20 – Movendo o kernel.bin para ./basic/boot

```
#!/bin/sh
qemu-system-i386 --kernel ./basic/boot/kernel.bin
```

Código 9 – Atualização do arquivo run.sh.

No arquivo grub.cfg, ajusta-se a execução deste SO como padrão (set default = 0), e o tempo aguardado para sua inicialização é ajustado em 0 seg (set timeout = 0). Além disso, é criada uma entrada de menu, para a escolha do SO, chamado de “basicOS”. Nessa entrada, define-se o dispositivo de boot como sendo o CD-ROM (set root='(hd96)'), carregado a partir do arquivo kernel.bin, o qual está localizado na pasta boot.

Agora iremos fazer a geração do ISO, para isto, criamos mais um script chamado make.sh (Codigo 10) no diretório mãe do projeto (Figura 21) que irá preparar o nosso arquivo ISO.

```
vboxuser@Debian12:~/Projeto_2/so$ ls
basic build.sh kasm.o kc.o kernel.asm kernel.c link.ld run.sh
vboxuser@Debian12:~/Projeto_2/so$ nano make.sh
vboxuser@Debian12:~/Projeto_2/so$ ls
basic build.sh kasm.o kc.o kernel.asm kernel.c link.ld make.sh run.sh
vboxuser@Debian12:~/Projeto_2/so$ ls -l make.sh
-rw-r--r-- 1 vboxuser vboxuser 45 Dec  7 04:54 make.sh
vboxuser@Debian12:~/Projeto_2/so$ chmod +rwx make.sh
vboxuser@Debian12:~/Projeto_2/so$ ls -l make.sh
-rwxr-xr-x 1 vboxuser vboxuser 45 Dec  7 04:54 make.sh
vboxuser@Debian12:~/Projeto_2/so$
```

Figura 21 – Criando script make.sh

```
#!/bin/sh
grub-mkrescue -o basic.iso basic
```

Código 10 – Conteúdo do arquivo make.sh.

Agora que está tudo certo com o nosso script, iremos executá-lo, e nisso observaremos que haverá criado um arquivo basic.iso com sucesso no nosso diretório raiz (Figura 22). Portanto, o código está dizendo ao grub para criar uma imagem ISO chamada basic.iso usando os arquivos presentes no diretório basic.

```
vboxuser@Debian12:~/Projeto_2/so$ ls
basic build.sh kasm.o kc.o kernel.asm kernel.c link.ld make.sh run.sh
vboxuser@Debian12:~/Projeto_2/so$ ./make.sh
xorriso 1.5.4 : RockRidge filesystem manipulator, libburnia project.

Drive current: -outdev 'stdio:basic.iso'
Media current: stdio file, overwriteable
Media status : is blank
Media summary: 0 sessions, 0 data blocks, 0 data, 48.4g free
Added to ISO image: directory '/'='/tmp/grub.YUqNmV'
xorriso : UPDATE : 335 files added in 1 seconds
Added to ISO image: directory '/'='/home/vboxuser/Projeto_2/so/basic'
xorriso : UPDATE : 339 files added in 1 seconds
xorriso : NOTE : Copying to System Area: 512 bytes from file '/usr/lib/grub/i386
-pc/boot_hybrid.img'
ISO image produced: 4638 sectors
Written to medium : 4638 sectors at LBA 0
Writing to 'stdio:basic.iso' completed successfully.

vboxuser@Debian12:~/Projeto_2/so$ ls
basic build.sh kc.o kernel.c make.sh
basic.iso kasm.o kernel.asm link.ld run.sh
vboxuser@Debian12:~/Projeto_2/so$
```

Figura 22 – ISO gerada com sucesso

1.4. Testando o projeto

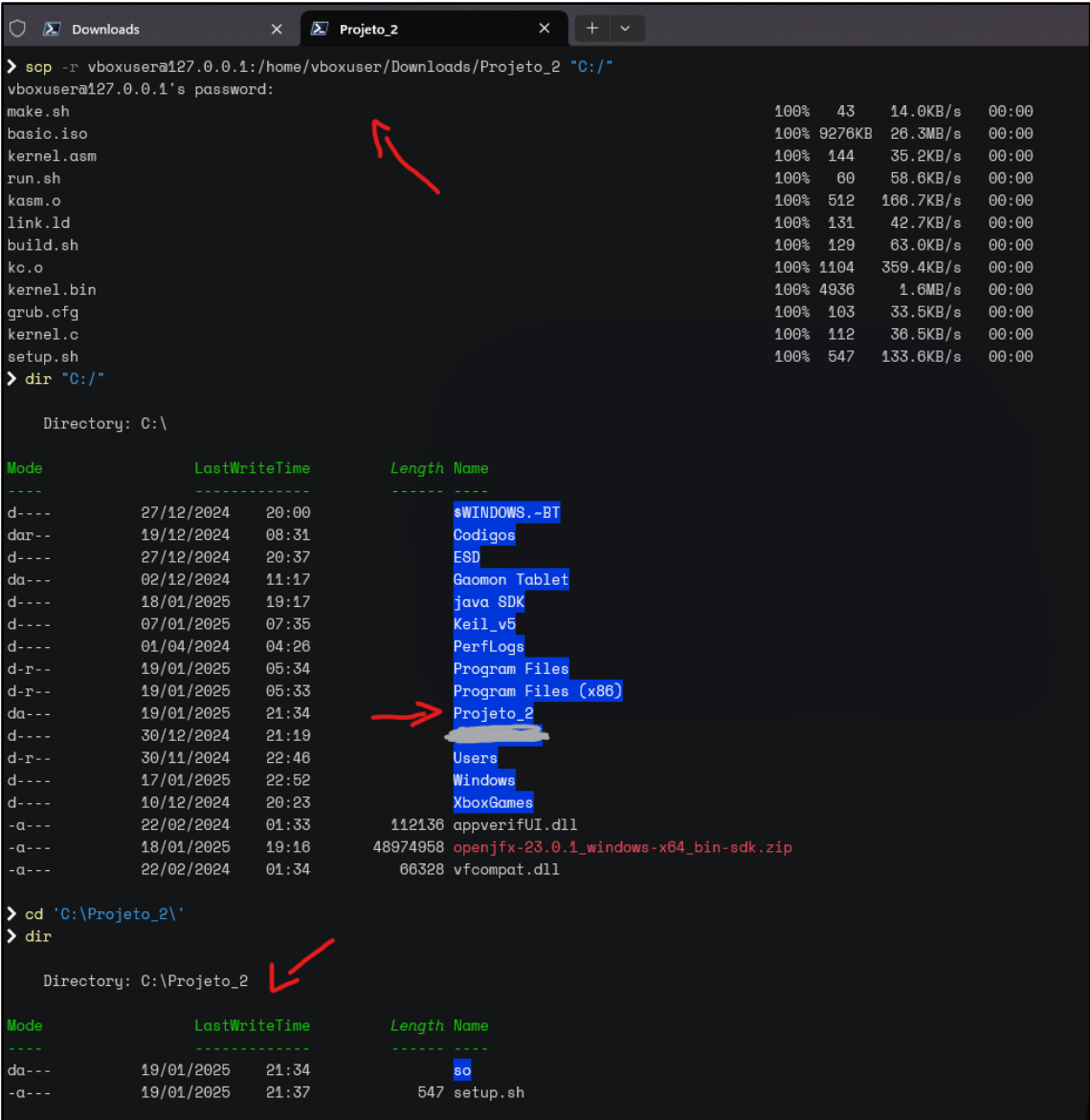
Neste projeto como eu havia dito no início, nosso hospedeiro é windows e estamos virtualizando um debian no virtualbox cujo todo esses procedimentos foram realizados nessa máquina virtual, sendo assim, como iremos testar outra máquina virtual, optamos por transferir a pasta raiz (Projeto_2) do guest para o hospedeiro utilizando SCP (Secure

Copy Protocol) visto anteriormente no projeto 1, para obtermos o arquivo ISO que geramos no final do tópico 1.3 – Preparação do projeto.

Sendo assim, não será necessário instalar o openssh server pois já temos tudo configurado e também os host já são considerados confiáveis (chaves autorizadas). Por outro lado, temos os seguintes IP: hospedeiro (localhost) e guest (localhost/127.0.0.1). A partir do hospedeiro, utilizaremos o prompt de comando no windows conforme o Código 11 para transferirmos os nossos arquivos.

```
scp -r vboxuser@127.0.0.1:/home/vboxuser/Downloads/Projeto_2 "C:/"
```

Código 11 – Transferir o diretório do Projeto_2.



```
> scp -r vboxuser@127.0.0.1:/home/vboxuser/Downloads/Projeto_2 "C:/"
vboxuser@127.0.0.1's password:
make.sh                                100% 43    14.0KB/s   00:00
basic.iso                              100% 9276KB 26.3MB/s   00:00
kernel.asm                             100% 144    35.2KB/s   00:00
run.sh                                 100% 60     58.6KB/s   00:00
kasm.o                                 100% 512    166.7KB/s   00:00
link.ld                                100% 131    42.7KB/s   00:00
build.sh                               100% 129    63.0KB/s   00:00
kc.o                                    100% 1104   359.4KB/s   00:00
kernel.bin                             100% 4936    1.6MB/s   00:00
grub.cfg                               100% 103    33.5KB/s   00:00
kernel.c                               100% 112    36.5KB/s   00:00
setup.sh                               100% 547    133.6KB/s   00:00
> dir "C:/"

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d---             27/12/2024    20:00             $WINDOWS.BT
dar--            19/12/2024     08:31             Codigos
d---             27/12/2024    20:37             ESD
da---            02/12/2024    11:17             Gaomon Tablet
d---            18/01/2025    19:17             java SDK
d---            07/01/2025     07:35             Keil_v5
d---            01/04/2024     04:26             PerfLogs
d-r--            19/01/2025     05:34             Program Files
d-r--            19/01/2025     05:33             Program Files (x86)
da---            19/01/2025    21:34             Projeto_2
d---            30/12/2024    21:19             Users
d---            17/01/2025    22:52             Windows
d---            10/12/2024    20:23             XboxGames
-a---            22/02/2024     01:33             112136 appverifUI.dll
-a---            18/01/2025    19:16             48974958 openjfx-23.0.1_windows-x64_bin-sdk.zip
-a---            22/02/2024     01:34             66328 vfcompat.dll

> cd "C:\Projeto_2\'
> dir

Directory: C:\Projeto_2

Mode                LastWriteTime         Length Name
----                -
da---            19/01/2025    21:34             so
-a---            19/01/2025    21:37             547 setup.sh
```

Figura 23 – Procedimento de transferência de arquivo SCP.

Após a transferência da pasta, para testar a nossa imagem ISO localizada em C:\Projeto_2\so\basic.iso, iremos criar uma máquina virtual no VirtualBox, com 64 MB

de memória RAM, 1 CPU, com descrições de “Other” para o tipo do SO e “Other/Unknown” para a versão (32 bits desconhecido) conforme a Figura 24.



Figura 24 – Criar máquina virtual.

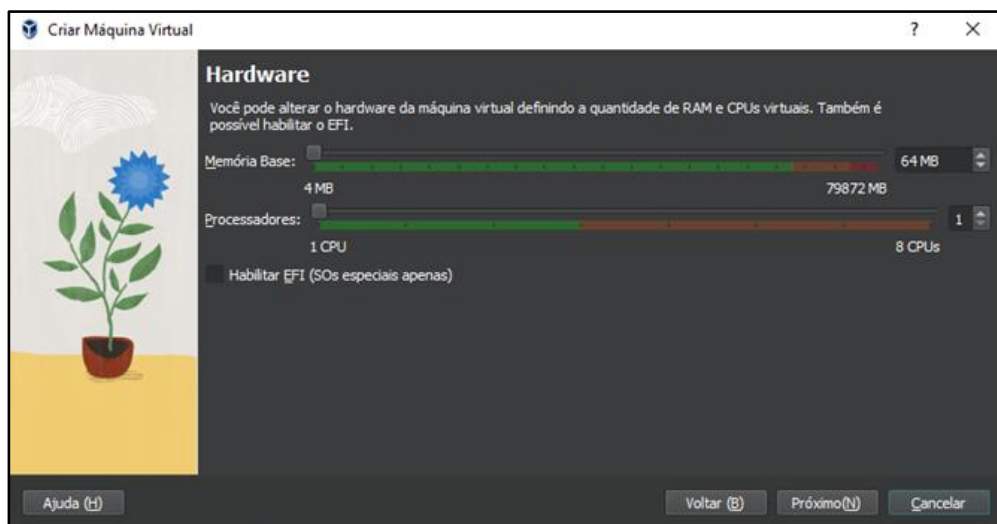


Figura 25 – Hardware Guest.

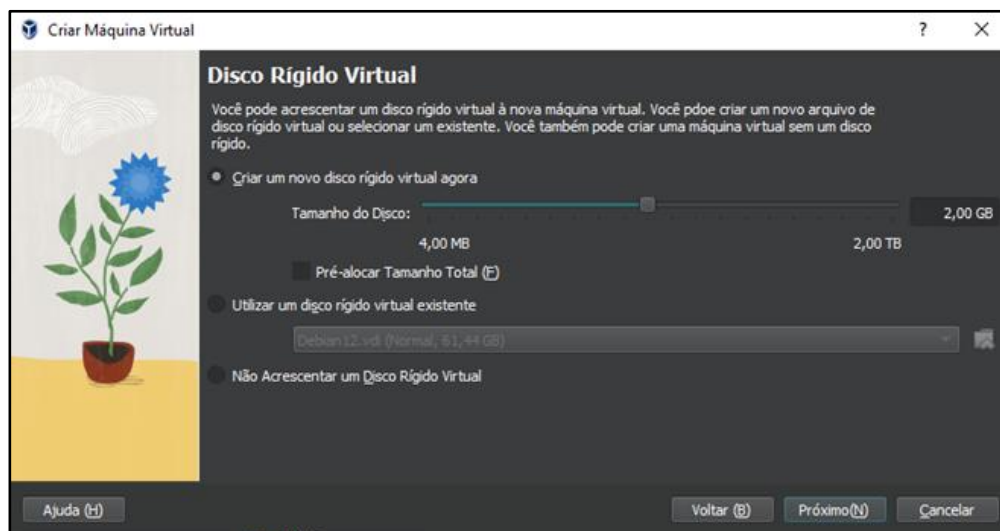


Figura 26 – Armazenamento Guest.

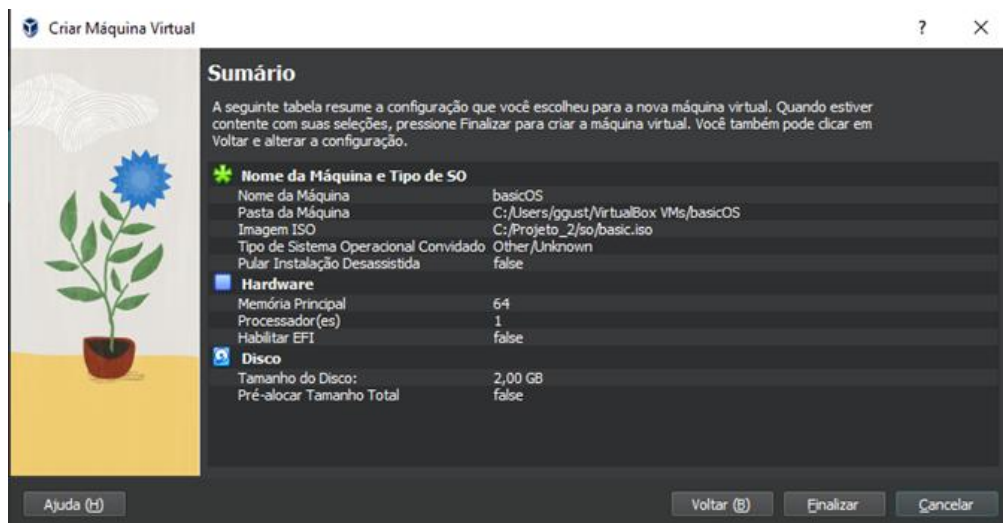


Figura 27 – Resumo.

Para finalizar, configuramos esta máquina virtual para ser inicializada com um CD de boot, apontando para a imagem ISO criada (simple.iso) conforme as próximas Figuras.

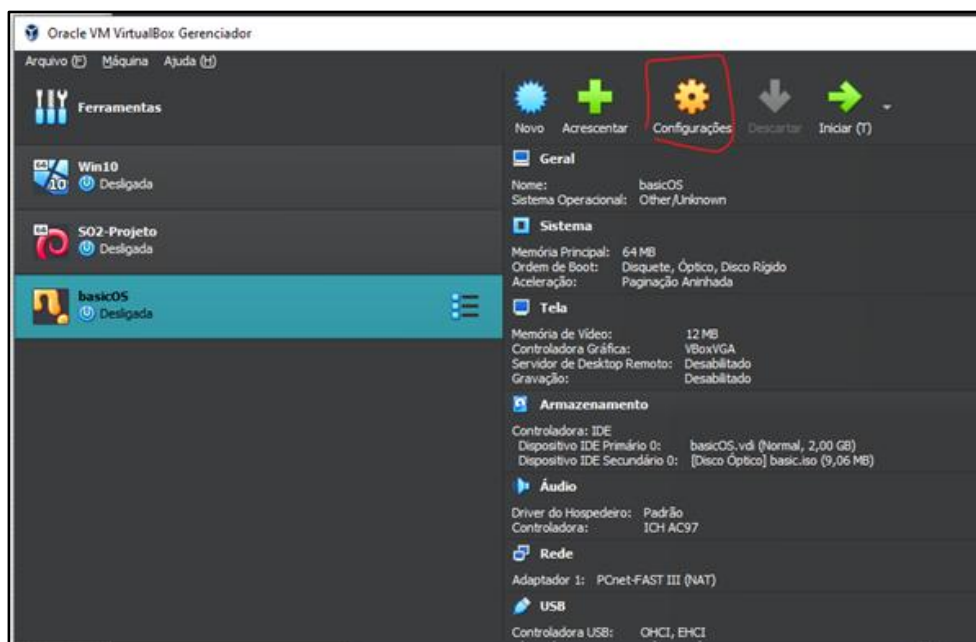


Figura 28 – Acessando a configuração da máquina virtual.

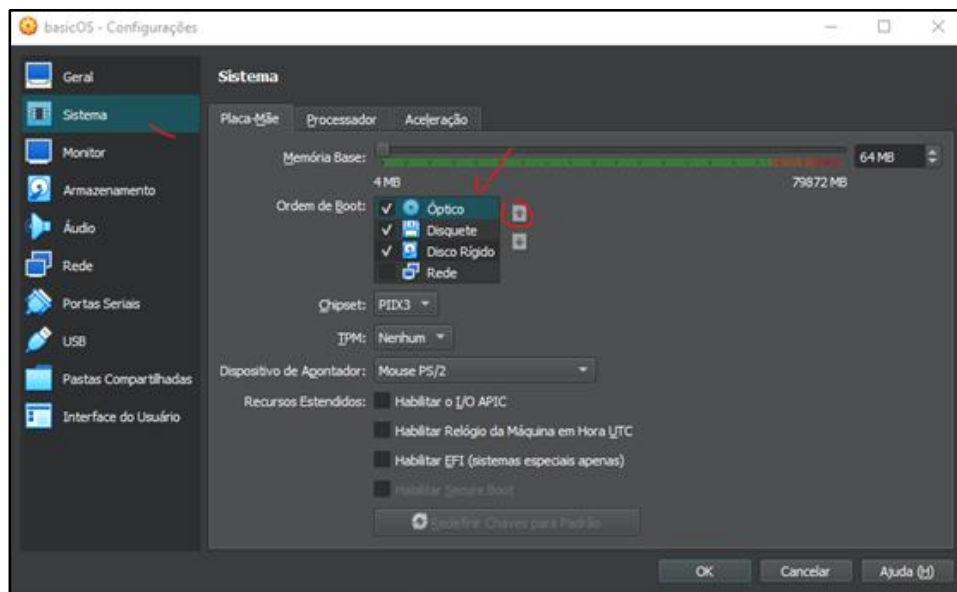


Figura 29 – Alterando a ordem de boot para inicializar com disco óptico (CD).

Após isto, basta clicar em ok e inicializar a máquina normalmente, cujo a mesma irá apresentar a tela inicial do grub que configuramos, basta pressionar Enter e irá inicializar o nosso primeiro sistema conforme a Figura 30.

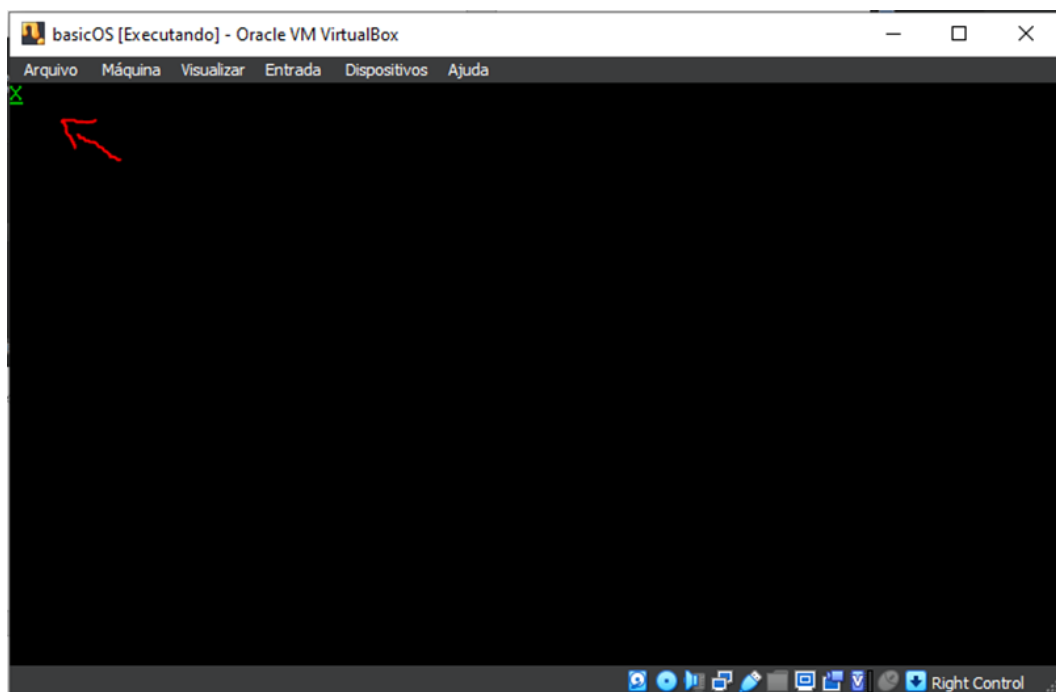


Figura 30 – Hospedeiro virtualizando um SO básico na VirtualBox, cujo a ISO desse SO foi criada/gerada/montada em uma outra máquina guest linux.

2. Tarefa 2: adicionando funcionalidades básicas ao kernel

2.1. Implementação do projeto

Com base na estrutura da prática de laboratório realizada na tarefa 1, baixamos uma ferramenta chamada tree (Figura 31) para visualizar melhor como ficou a organização.

```
vboxuser@Debian12:~$ tree Projeto_2
Projeto_2
├── setup.sh
└── so
    ├── basic
    │   ├── boot
    │   │   ├── grub
    │   │   └── grub.cfg
    │   └── kernel.bin
    ├── basic.iso
    ├── build.sh
    ├── kasm.o
    ├── kc.o
    ├── kernel.asm
    ├── kernel.c
    ├── link.ld
    ├── make.sh
    └── run.sh

5 directories, 12 files
```

Figura 31 – Utilização da ferramenta Tree

Agora iremos criar um subdiretório chamado “include” dentro da pasta “basic” do Projeto_2. Vale ressaltar que, estamos criando este subdiretório nesse local pois estamos seguindo uma convenção comum utilizada em projetos de programação, onde os cabeçalhos e as bibliotecas frequentemente são colocados em subdiretórios chamados include.

Por seguinte, dentro desta pasta, serão criados quatro arquivos: types.h, string.h, system.h e screen.h, cujo o objetivo destes arquivos será de permitir a incorporação de novas funções ao kernel do SO. Para criação do arquivo utilizamos a ferramenta touch que já tem instalado no linux para criarmos arquivos vazios (Figura 32) que serão explicados posteriormente para uma melhor compreensão.

```
vboxuser@Debian12:~$ cd Projeto_2/so/basic
vboxuser@Debian12:~/Projeto_2/so/basic$ mkdir include
vboxuser@Debian12:~/Projeto_2/so/basic$ cd include
vboxuser@Debian12:~/Projeto_2/so/basic/include$ touch screen.h string.h system.h types.h
vboxuser@Debian12:~/Projeto_2/so/basic/include$ ls
screen.h string.h system.h types.h
```

Figura 32 – Criação do include e arquivos vazios

A partir de agora, as funções serão “habilitadas”, como por exemplo uma função print, a partir do meio da inclusão de bibliotecas mediante o uso da diretiva #include.

Sendo assim, primeiramente criaremos o arquivo types.h (Código 12) que servirá para definir os tipos de dados que serão usados nos demais arquivos de biblioteca.

```
#ifndef TYPES_H
#define TYPES_H

typedef signed char int8;
typedef unsigned char uint8;

typedef signed short int16;
typedef unsigned short uint16;

typedef signed int int32;
typedef unsigned int uint32;

typedef signed long int64;
typedef unsigned long uint64;

typedef char* string;

#endif
```

Código 12 – Conteúdo do arquivo types.h.

As duas primeiras linhas especificam a definição do arquivo de biblioteca. A inserção destas linhas (incluindo o #endif ao final) não é uma obrigatoriedade, mas faz parte das boas práticas de codificação de bibliotecas como por exemplo ao usar a ferramenta CodeBlocks, isso é padronizado ao se criar um arquivo de biblioteca. Na sequência, são definidos alguns tipos inteiros (de 8, 16, 32 e 64 bits) a partir de tipos primitivos de dados. Também foi criado um novo tipo chamado string, o qual representa uma cadeia de caracteres sem tamanho definido, a partir de um ponteiro para um tipo de dado primitivo caractere.

Seguindo a sequência de definição para a biblioteca do sistema operacional, é lógico especificar as funções para manipulação de strings. Portanto, a próxima etapa é a criação do arquivo string.h (Código 13).

```
#ifndef STRING_H
#define STRING_H
#include "types.h"

uint16 strlen(string ch) {
    uint16 i = 1;
    while(ch[i++]);
    return --i;
}

#endif
```

Código 13 – Conteúdo do arquivo string.h.

Por enquanto, este arquivo inclui apenas a função uint16 strlen(string) que dependerá dos tipos uint16 e string, os quais foram definidos em types.h. Além disso, essa função retorna um uint16 (inteiro sem sinal de 16 bits) e recebe como parâmetro uma string ch, logo, essa função uint16 strlen (string ch) é projetada para calcular o

comprimento de uma string em termos de número de caracteres, no qual é realizado um loop while para iterar através dos caracteres da string ch até encontrar o caractere nulo ('\0'), que marca o final da string em C. O contador i é incrementado durante cada iteração do loop. No final, a função retorna o valor decrementado de i, representando o comprimento total da string (excluindo o caractere nulo).

Agora, o próximo arquivo criado é o system.h (Código 14).

```
#ifndef SYSTEM_H
#define SYSTEM_H
#include "types.h"

uint8 inportb(uint16 _port) {
    uint8 rv;
    __asm__ __volatile__ ("inb %1, %0"
        : "=a" (rv) /* operando de saída */
        : "dN" (_port)); /* operando de entrada */
    return rv;
}

void outportb(uint16 _port, uint8 _data) {
    __asm__ __volatile__ ("outb %1, %0"
        : /* operando de saída */
        : "dN" (_port), "a" (_data)); /* operando de entrada */
}

#endif
```

Código 14 – Conteúdo do arquivo system.h.

O Código 14 apresenta duas funções, no qual, uma retorna um valor char (uint8) e a outra não retorna valor (void). Para que possamos usar os tipos de dados que definimos anteriormente em types.h, incluímos este arquivo. A primeira função, uint8 inportb(uint16) recebe como parâmetro um inteiro de 16 bits e devolve um inteiro de 8 bits. O parâmetro de entrada é um endereço de porta (_port) de 16 bits. Internamente a função, a grosso modo, a variável rv será conectada a porta _port e o valor contido na porta será inserido nesta variável.

Vale ressaltar que, essas duas funções inportb e outportb são de origens do assembly, portanto, sua utilidade está relacionada com o controle da posição do cursor que irá aparecer em nossa tela por exemplo, ou seja, é necessário “falar” diretamente com as portas. Como existem dispositivos com os quais se deseja conectar, como ler/escrever dados a partir deles, é necessário se usar portas. Na prática, o Código 14 é muito importante porque não é possível se falar com portas diretamente usando a linguagem C e é por essa razão que foi criada uma função na linguagem Assembly.

Além disso, em alguns dispositivos, como a tela (screen), é possível efetuar a comunicação bastando-se escrever dados para uma área específica da memória (RAM). Esta continuará atualizando seus dados dependendo do que está guardado naquela área particular de memória. No caso do cursor, mouse e teclado, é necessário de fato se falar diretamente com as portas. Assim a função uint8 inportb(uint16) irá ler a partir da porta e a função void outportb(uint16, uint8) irá escrever dados na porta. No caso da função

outportb(uint16, uint8) o primeiro argumento é a porta desejada e o segundo argumento são os dados que serão escritos para aquela porta particular.

Agora, trataremos dos aspectos mais técnicos do Código 14. Os prefixos e sufixos “__” (dois sublinhados) não chegam a ser necessários, entretanto, podem prevenir conflitos de nomes em programas. A notação apresentada é o que se chama de extended asm ou Assembly estendido. Esse é um código Assembly que pode ser usado diretamente dentro de código C para fins de acesso a registradores ou acesso a instruções de baixo nível.

O código assembly estendido permite a especificação de registradores de entrada, de saída e clobbered (que podem ser sobrescritos durante a execução). No Código 14, este foi indentado de modo a demonstrar cada uma das linhas de entrada/saída/clobbered (não usado). A primeira linha que começa com um ‘:’ especifica os operandos de saída, a segunda linha indica os operandos de entrada e , caso existisse, a terceira linha especificaria os operandos clobbered. ‘a’ é um exemplo de constante [5]. Constantes de saída devem possuir como prefixo um ‘=’, como é o caso do “=a” (= é um modificador indicando write only). Constantes de entrada e saída devem possuir seu argumento em C correspondente entre parêntesis (não vale para a linha de parâmetro clobbered). Note que o uso de “0” e “1” são usados para assegurar que quando uma mesma variável é indicada em mais de um lugar no **asm** estendido, esta variável é mapeada apenas para um registrador. Em caso de mais de uma referência a mesma variável, o compilador pode ou não colocá-la no mesmo registrador já atribuído.

As funções inb e outb são funções __asm__ para a realização de acesso a entrada e saída (I/O) [6]. No Código 14, na linha de código __asm__ estendido especificando inportb, é possível verificar a leitura do valor a partir de _port e a atribuição deste a rv. A diretiva __asm__ __volatile__ foi usada devido aos mecanismos de otimização do compilador. Graças a essas otimizações, o código pode ou não aparecer na localidade especificada pelo programador. Para evitar isso, usa-se essa diretiva, ao invés de simplesmente __asm__.

Agora, prosseguindo com a criação dos arquivos, temos o screen.h (Código 15) que introduz as principais funções para manipulação de caracteres na tela do computador. Essas funções são previstas para monitores VGA, que é o padrão usado durante o boot de qualquer sistema operacional (Figura 33).

0xb8000				0xb8002								
	0	1	2	3					156	157	158	159
0	'H'	0x04	'E'	0x02								
1												
					Colunas = 80 (cada uma é dividida em 2 bytes, um para caractere e outro para cor). Linhas = 25							
24												

Figura 32 – Definições para um monitor VGA [3].

Na Figura 32 é possível ver o endereço da primeira posição na tela (0xb8000). Além disso, é possível verificar que cada posição na tela é definida por dois bytes: o primeiro define o caractere a ser impresso e o segundo define a sua cor. São definidas ao todo 25 linhas por 80 colunas. Os códigos da biblioteca screen.h apresentados no Código 15, levam em consideração as definições da Figura 32.

A biblioteca screen.h (Código 15) é a consagração de todo o trabalho realizado até o momento. Esta biblioteca utiliza as demais bibliotecas implementadas até então (types.h, system.h e string.h). As principais variáveis são cursorX e cursorY, as quais representam a posição do cursor na tela (nos eixos x e y). Além disso, as constantes sw, sh e sd representam a largura, altura e profundidade da tela. Conquanto, se compararmos a Figura 32 que a largura é equivalente a quantidade de colunas na tela, a altura é equivalente a quantidade de linhas e a profundidade representa a definição de cada caractere na tela (valor e cor).

Para a biblioteca screen.h foram definidas as funções void clearLine(uint8, uint8), void updateCursor(void), void clearScreen(void), void scrollUp(uint8), void newLineCheck(void), void printch(char) e void print(string). Além disso, no código abaixo, de acordo com a tarefa, foi solicitado a inserção de comentários detalhados para as funções do código 15, portanto, o código já está com os comentários incluídos.

```
#ifndef SCREEN_H
#define SCREEN_H
#include "types.h"
#include "system.h"
#include "string.h"
int32 cursorX = 0, cursorY = 0; // Posição do cursor na tela
const uint8 sw = 80, sh = 25, sd = 2; // Dimensões da tela (largura, altura, profundidade)

// Limpa a tela de uma linha específica até outra
void clearLine(uint8 from, uint8 to) {
    uint16 i = sw * from * sd;
    string vidmem = (string)0xb8000;
    for (i; i < (sw * (to + 1) * sd); i++) {
        vidmem[i] = 0x0; // Valor (caractere) 0
        vidmem[i+1] = 0x0F; // Cor branca sobre fundo preto
    }
}

// Atualiza a posição do cursor na tela
void updateCursor() {
    unsigned temp;
    temp = cursorY * sw + cursorX;
    outportb(0x3D4, 14);
    outportb(0x3D5, temp >> 8);
    outportb(0x3D4, 15);
    outportb(0x3D5, temp);
}

// Limpa toda a tela
void clearScreen() {
    clearLine(0, sh-1); // Chama clearLine p/ limpar da linha 0 até n (sh-1)
    cursorX = 0; // Reseta a posição do cursor na largura
    cursorY = 0; // Reseta a posição do cursor na altura
    updateCursor(); // Atualiza a posição do cursor
}
```

```

}

// Desloca conteúdo da tela para cima, remove 'lineNumber' linhas na parte superior
void scrollUp(uint8 lineNumber) {
    string vidmem = (string)0xb8000;
    uint16 i = 0;

    for (i; i < sw * (sh - 1) * sd; i++) {
        vidmem[i] = vidmem[i + sw * sd * lineNumber]; // Move o conteúdo da tela p/ cima
    }

    clearLine(sh - 1 - lineNumber, sh - 1); // Limpa linhas recém-criadas na parte inferior
    if ((cursorY - lineNumber) < 0) {
        cursorY = 0;
        cursorX = 0;
    } else {
        cursorY -= lineNumber; // Atualiza a posição do cursor na altura
    }
    updateCursor(); // Atualiza a posição do cursor
}

// Verifica se o cursor atingiu a última linha da tela, e se sim, rola a tela para cima
void newLineCheck() {
    if (cursorY >= sh-1) {
        scrollUp(1); // Chama scrollUp p/ rolar a tela p/ cima removendo uma linha
    }
}

// Imprime um caractere na posição atual do cursor
void printch(char c) {
    string vidmem = (string)0xb8000;
    switch(c) {
        case (0x08): // Retrocesso (BACKSPACE)
            if(cursorX > 0) {
                cursorX--; // Move cursor p/ posição anterior na largura
                vidmem[(cursorY * sw + cursorX)*sd] = 0x00; // Define o caractere como nulo
            } break;
        case (0x09): // Representa a tecla TAB
            cursorX = (cursorX + 8) & ~(8 - 1); // Move cursor p/ a próxima posição TAB
            break;
        case ('\r'): // Retorno de Carro
            cursorX = 0; // Move o cursor para a primeira posição na largura
            break;
        case ('\n'): // Nova Linha
            cursorX = 0; // Move o cursor para a primeira posição na largura
            cursorY++; // Move o cursor para a próxima linha na altura
            break;
        default:
            vidmem[((cursorY * sw + cursorX)) * sd] = c; // Define o caractere na tela
            vidmem[((cursorY * sw + cursorX)) * sd + 1] = 0x0F; // Define a cor do caractere
            cursorX++; // Move o cursor para a próxima posição na largura
            break;
    }
    if (cursorX >= sw) {
        cursorX = 0; // Se ultrapassar a largura, move o cursor para a próxima linha
        cursorY++; // na altura
    }
    newLineCheck(); // Verifica se o cursor atingiu a última linha da tela
    updateCursor(); // Atualiza a posição do cursor
}

```

```

// Imprime uma string chamando repetidamente printch para cada caractere na string
void print(string ch) {
    uint16 i = 0;
    for(i; i < strlen(ch); i++) {
        printch(ch[i]); // Chama printch para cada caractere na string
    }
} #endif

```

Código 15 – Conteúdo do arquivo screen.h.

Agora iremos explicar brevemente cada função do código 15 para partirmos para os testes.

A função **void clearLine(uint8, uint8)**, limpa a tela a partir de uma linha específica até outra linha específica. Esta função utiliza uma variável auxiliar *i*, que define o ponto de partida em função de largura (i.e. colunas) e profundidade, para a posição pretendida (from). A variável *vidmem* é uma string a qual é inicializada com o valor 0xb8000 (endereço da primeira posição válida na tela). Como se trata de um endereço numérico, faz-se um casting por segurança. Na sequência, todas as posições de 0 até *n* (*to*+1) em valor e cor (valor 0 e cor preta, também 0).

A função **void updateCursor(void)**, envia a localização do cursor para uma porta específica (0x3D4 e 0x3D5). Estas portas são padronizadas para a arquitetura x86 e controlam a posição do cursor na tela em modo VGA (modo usando 80 colunas x 25 linhas x 16 cores). Assim, para mudar a posição do cursor, basta atualizar os valores de *x* e *y* e então chamar a função **void updateCursor(void)**. Explicando esta função um pouco melhor, a variável *temp* recebe a posição em função dos eixos *x* e *y*. A função **outportb()** já foi explicada, e em **outportb(0x3D4, 14)**, seleciona a localização do cursor no registrador de controle (Control Register ou CRT). Em **outportb(0x3D5, temp >> 8)**, ela envia o byte de alta no barramento. Em **outportb(0x3D4, 15)**, seleciona o envio do byte de baixa no CRT e em **outportb(0x3D5, temp)** envia o byte de baixa da localização do cursor no barramento.

A função **void clearScreen(void)** faz a limpeza da tela da primeira à última linha e atualiza a posição do cursor nos eixos *x* e *y* para 0. A função **void scrollUp(uint8)** desloca o texto que aparece na tela *lineNumber* linhas. A função **void printch(char)** imprime um caractere na tela e a função **void print(string)** imprime uma string na tela. A função **void newLineCheck(void)** é uma função de verificação para confirmar se alcançamos a última linha na tela, tomando as medidas cabíveis.

A função **void scrollUp(uint8 lineNumber)** desloca o conteúdo da tela para cima, removendo a quantidade especificada de linhas na parte superior, e limpa as linhas recém-criadas na parte inferior. E por último, atualize as posições do cursor conforme necessário.

A função **void newLineCheck()** verifica se o cursor atingiu a última linha da tela (*sh*-1). Se verdadeiro, chama **scrollUp(1)** para rolar a tela para cima.

A função **void printch(char c)** realiza a impressão de um caractere na posição atual do cursor e faz o tratamento dos caracteres especiais como retrocesso (0x08), TAB (0x09), retorno de carro ('\r'), e nova linha ('\n'). E por último também atualiza as posições do cursor após a impressão.

A função **void print(string ch)** imprime uma string que chama repetidamente printch para cada caractere na string.

Agora iremos colocar os códigos nos respectivos arquivos para prosseguirmos conforme a Figura 33.

```
vboxuser@Debian12:~/Projeto_2/so/basic/include$ ls
screen.h string.h system.h types.h
vboxuser@Debian12:~/Projeto_2/so/basic/include$ nano screen.h
vboxuser@Debian12:~/Projeto_2/so/basic/include$ nano system.h
vboxuser@Debian12:~/Projeto_2/so/basic/include$ nano string.h
vboxuser@Debian12:~/Projeto_2/so/basic/include$ nano types.h
vboxuser@Debian12:~/Projeto_2/so/basic/include$ tree /home/vboxuser/Projeto_2
/home/vboxuser/Projeto_2
├── setup.sh
├── so
│   ├── basic
│   │   ├── boot
│   │   │   ├── grub
│   │   │   │   └── grub.cfg
│   │   │   └── kernel.bin
│   │   └── include
│   │       ├── screen.h
│   │       ├── string.h
│   │       ├── system.h
│   │       └── types.h
│   ├── basic.iso
│   ├── build.sh
│   ├── kasm.o
│   ├── kc.o
│   ├── kernel.asm
│   ├── kernel.c
│   ├── link.ld
│   ├── make.sh
│   └── run.sh
└── 6 directories, 16 files
```

Figura 33 – Colando os códigos e verificando a estrutura final

2.2. Teste da função updateCursor() no arquivo kernel.c

Com base no que foi dito no Tópico 2.1. Implementação do projeto, iremos agora realizar a tarefa proposta pelo professor, de realizarmos um teste utilizando a função updateCursor() no arquivo kernel.c, no qual terá de ser realizado novamente a compilação e execução do trabalho utilizando-se o QEMU e verificar o resultado. Vale ressaltar que deverá ser utilizado os valores de cursorX = 20 e cursorY = 15 para testes.

Sendo assim, de início, iremos alterar o nosso arquivo kernel.c localizado em Projeto_2/so de modo que seja utilizado a função UpdateCursor() conforme o código 16 abaixo atualizado.

```
vboxuser@Debian12:~/Projeto_2/so$ ls
basic      build.sh  kc.o      kernel.c  make.sh
basic.iso  kasm.o   kernel.asm link.ld   run.sh
vboxuser@Debian12:~/Projeto_2/so$ nano kernel.c
vboxuser@Debian12:~/Projeto_2/so$
```

Figura 34 – Editando o arquivo kernel.c

```
kmain()
{
    char *vidmem = (char*) 0xb8000;
    vidmem[0] = 'X';
    vidmem[1] = 0x02; /* cor do caractere: verde */

    // Chama a função updateCursor() com valores específicos
    cursorX = 20;
    cursorY = 15;
    updateCursor();
}
```

Código 16 – Atualização do conteúdo do arquivo kernel.c.

Agora iremos realizar a compilação do nosso projeto seguindo os mesmos passos que realizamos no tópico 1.3 - Preparação do projeto - Figura 14.

```
vboxuser@Debian12:~/Projeto_2/so$ ./build.sh
kernel.c: In function 'kmain':
kernel.c:8:5: error: 'cursorX' undeclared (first use in this function)
   8 |     cursorX = 20;
     |     ^~~~~~
kernel.c:8:5: note: each undeclared identifier is reported only once for each function it appears in
kernel.c:9:5: error: 'cursorY' undeclared (first use in this function)
   9 |     cursorY = 15;
     |     ^~~~~~
ld: warning: kasm.o: missing .note.GNU-stack section implies executable stack
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
ld: warning: kernel.bin has a LOAD segment with RWX permissions
```

Figura 35 – Compilação error.

Infelizmente tivemos esse erro, mas é óbvio que iria dar errado, pois não declaramos corretamente as funções, sendo assim, partindo como ponto a observação final do professor, vale destacar que todas as funções declaradas são consideradas “estáticas”. Nesse contexto, "estático" significa que as funções são definidas diretamente no arquivo de cabeçalho (screen.h). A recomendação é que essas funções sejam **declaradas com "extern"** no arquivo de **cabeçalho e implementadas em um arquivo separado** (por

exemplo, screen.c). Essa abordagem evita a duplicação de código quando a função é chamada várias vezes no kernel, melhorando a organização e o tamanho do executável final. Para implementar isso, você precisa criar um arquivo .c correspondente (por exemplo, screen.c) e adicionar a compilação desse arquivo no script build.sh.

Certo, então a primeira coisa que iremos fazer é criar um arquivo novo separado chamado de screen.c no mesmo diretório onde está o kernel.c (./Projeto_2/so) e iremos implementar as funções de screen.h, system.h, types.h e string.h que havíamos, logo, formaremos novos abaixo.

```
#include "basic/include/screen.h"

int32 cursorX = 0, cursorY = 0; // Posição do cursor na tela
const uint8 sw = 80, sh = 25, sd = 2; // Dimensões da tela (largura, altura, profundidade)

uint16 strlenh(string ch) {
    uint16 i = 1;
    while(ch[i++]);
    return --i;
}

uint8 inportb (uint16 _port) {
    uint8 rv;
    __asm__ __volatile__ ("inb %1, %0"
                          : "=a" (rv)                /* operando de saída */
                          : "dN" (_port)); /* operando de entrada */
    return rv;
}

void outportb (uint16 _port, uint8 _data) {
    __asm__ __volatile__ ("outb %1, %0"
                          :                               /* operando de saída */
                          : "dN" (_port), "a" (_data)); /* operando de entrada */
}

// Limpa a tela de uma linha específica até outra
void clearLine(uint8 from, uint8 to) {
    uint16 i = sw * from * sd;
    string vidmem = (string)0xb8000;
    for (i; i < (sw * (to + 1) * sd); i++) {
        vidmem[i] = 0x0; // Valor (caractere) 0
        vidmem[i+1] = 0x0F; // Cor branca sobre fundo preto
    }
}

// Atualiza a posição do cursor na tela
void updateCursor() {
    unsigned temp;
    temp = cursorY * sw + cursorX;
    outportb(0x3D4, 14);
    outportb(0x3D5, temp >> 8);
    outportb(0x3D4, 15);
    outportb(0x3D5, temp);
}

// Limpa toda a tela
void clearScreen() {
    clearLine(0, sh-1); // Chama clearLine p/ limpar da linha 0 até n (sh-1)
}
```

```

        cursorX = 0;    // Reseta a posição do cursor na largura
        cursorY = 0;    // Reseta a posição do cursor na altura
        updateCursor(); // Atualiza a posição do cursor
    }

    // Desloca conteúdo da tela para cima, remove 'lineNumber' linhas na parte superior
    void scrollUp(uint8 lineNumber) {
        string vidmem = (string)0xb8000;
        uint16 i = 0;

        for (i; i < sw * (sh - 1) * sd; i++) {
            vidmem[i] = vidmem[i + sw * sd * lineNumber]; // Move o conteúdo da tela p/ cima
        }

        clearLine(sh - 1 - lineNumber, sh - 1); // Limpa linhas recém-criadas na parte inferior
        if ((cursorY - lineNumber) < 0) {
            cursorY = 0;
            cursorX = 0;
        } else {
            cursorY -= lineNumber; // Atualiza a posição do cursor na altura
        }
        updateCursor(); // Atualiza a posição do cursor
    }

    // Verifica se o cursor atingiu a última linha da tela, e se sim, rola a tela para cima
    void newLineCheck() {
        if (cursorY >= sh-1) {
            scrollUp(1); // Chama scrollUp p/ rolar a tela p/ cima removendo uma linha
        }
    }

    // Imprime um caractere na posição atual do cursor
    void printch(char c) {
        string vidmem = (string)0xb8000;
        switch(c) {
            case (0x08): // Retrocesso (BACKSPACE)
                if(cursorX > 0) {
                    cursorX--; // Move cursor p/ posição anterior na largura
                    vidmem[(cursorY * sw + cursorX)*sd] = 0x00; // Define o caractere como nulo
                } break;
            case (0x09): // Representa a tecla TAB
                cursorX = (cursorX + 8) & ~(8 - 1); // Move cursor p/ a próxima posição TAB
                break;
            case ('\r'): // Retorno de Carro
                cursorX = 0; // Move o cursor para a primeira posição na largura
                break;
            case ('\n'): // Nova Linha
                cursorX = 0; // Move o cursor para a primeira posição na largura
                cursorY++; // Move o cursor para a próxima linha na altura
                break;
            default:
                vidmem[((cursorY * sw + cursorX)) * sd] = c; // Define o caractere na tela
                vidmem[((cursorY * sw + cursorX)) * sd + 1] = 0x0F; // Define a cor do caractere
                cursorX++; // Move o cursor para a próxima posição na largura
                break;
        }
        if (cursorX >= sw) {
            cursorX = 0; // Se ultrapassar a largura, move o cursor para a próxima linha
            cursorY++; // na altura
        }

        newLineCheck(); // Verifica se o cursor atingiu a última linha da tela
    }

```



```

        updateCursor(); // Atualiza a posição do cursor
    }

    // Imprime uma string chamando repetidamente printch para cada caractere na string
    void print(string ch) {
        uint16 i = 0;
        for(i; i < strlen(ch); i++) {
            printch(ch[i]); // Chama printch para cada caractere na string
        }
    }
}

```

Código 17 – Conteúdo do arquivo screen.c.

Agora, em nosso arquivo string.h localizado na pasta ./Projeto_2/so/basic/include, declare as variáveis como externas conforme o código 18 abaixo.

```

#ifndef STRING_H
#define STRING_H
#include "types.h"

uint16 strlen(string ch);

#endif

```

Código 18 – Atualização do conteúdo do arquivo string.h.

Agora, em nosso arquivo system.h localizado na pasta ./Projeto_2/so/basic/include, declare as variáveis como externas conforme o código 19 abaixo.

```

#ifndef SYSTEM_H
#define SYSTEM_H
#include "types.h"

uint8 inportb (uint16 _port);
void outportb (uint16 _port, uint8 _data);

#endif

```

Código 19 – Atualização do conteúdo do arquivo system.h.

Agora, em nosso arquivo types.h localizado na pasta ./Projeto_2/so/basic/include, declare as variáveis como externas conforme o código 20 abaixo.

```

#ifndef TYPES_H
#define TYPES_H

typedef signed char int8;
typedef unsigned char uint8;

typedef signed short int16;
typedef unsigned short uint16;

typedef signed int int32;
typedef unsigned int uint32;

```

```
typedef signed long int64;
typedef unsigned long uint64;

typedef char* string;

#endif
```

Código 20 – Atualização do conteúdo do arquivo types.h.

Agora, em nosso arquivo screen.h localizado na pasta ./Projeto_2/so/basic/include, declare as variáveis como externas conforme o código 21 abaixo.

```
#ifndef SCREEN_H
#define SCREEN_H

#include "types.h"
#include "system.h"
#include "string.h"

extern int32 cursorX;
extern int32 cursorY;

#endif
```

Código 21 – Atualização do conteúdo do arquivo screen.h.

Isso indica que as variáveis cursorX e cursorY são definidas em outro lugar (no arquivo screen.c, neste caso). Agora, em nosso arquivo kernel.c, vamos apenas precisar de incluir o cabeçalho e chamar as funções conforme necessário, como podemos ver no código 22. Além disso, utilizaremos os valores de cursorX = 20 e cursorY = 15 para testes.

```
#include "basic/include/screen.h"

void kmain() {
    // Chama a função updateCursor() com valores específicos
    cursorX = 20;
    cursorY = 15;
    updateCursor();
}
```

Código 22 – Atualização do conteúdo do arquivo kernel.c.

Vale ressaltar que, anteriormente na tarefa 1, havíamos feito com que o kernel.c escreve-se um caractere “X” diretamente no endereço de memória de vídeo (‘0xb8000’), porém não será mais necessário e optamos por removê-la já que iremos utilizar o updateCursor() para manipular o cursor na tela.

E agora também editaremos o nosso script build.sh para conseguirmos compilar tudo sem problemas.

```
#!/bin/sh
nasm -f elf32 kernel.asm -o kasm.o
gcc -m32 -c -w kernel.c -o kc.o
gcc -m32 -c -w screen.c -o screen.o
ld -m elf_i386 -T link.ld -o kernel.bin kasm.o screen.o kc.o
```

Código 23 – Atualização do conteúdo do arquivo build.sh.

E por último temos a visão geral de como ficou para prosseguirmos para compilação e testes.

```
vboxuser@Debian12:~/Projeto_2/so$ nano screen.c
vboxuser@Debian12:~/Projeto_2/so$ nano ./basic/include/screen.h
vboxuser@Debian12:~/Projeto_2/so$ nano kernel.c
vboxuser@Debian12:~/Projeto_2/so$ tree /home/vboxuser/Projeto_2
/home/vboxuser/Projeto_2
├── setup.sh
├── so
│   ├── basic
│   │   ├── boot
│   │   │   ├── grub
│   │   │   │   └── grub.cfg
│   │   │   └── kernel.bin
│   │   └── include
│   │       ├── screen.h
│   │       ├── string.h
│   │       ├── system.h
│   │       └── types.h
│   ├── basic.iso
│   ├── build.sh
│   ├── kasm.o
│   ├── kc.o
│   ├── kernel.asm
│   ├── kernel.bin
│   ├── kernel.c
│   ├── link.ld
│   ├── make.sh
│   ├── run.sh
│   ├── screen.c
│   └── screen.o
└── 6 directories, 19 files
```

Figura 36 – Editando, criando arquivos e a visão geral da estrutura.

Agora que estamos com tudo preparado, vamos realizar novamente a compilação dos códigos feitos anteriormente utilizando o script build.sh.

```
vboxuser@Debian12:~/Projeto_2/so$ ./build.sh
ld: warning: kasm.o: missing .note.GNU-stack section implies executable stack
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
ld: warning: kernel.bin has a LOAD segment with RWX permissions
vboxuser@Debian12:~/Projeto_2/so$
```

Figura 37 – Compilado com sucesso.

Agora vamos mover o arquivo gerado (kernel.bin) para a pasta ./basic/boot e depois vamos testar utilizando nosso script run.sh.

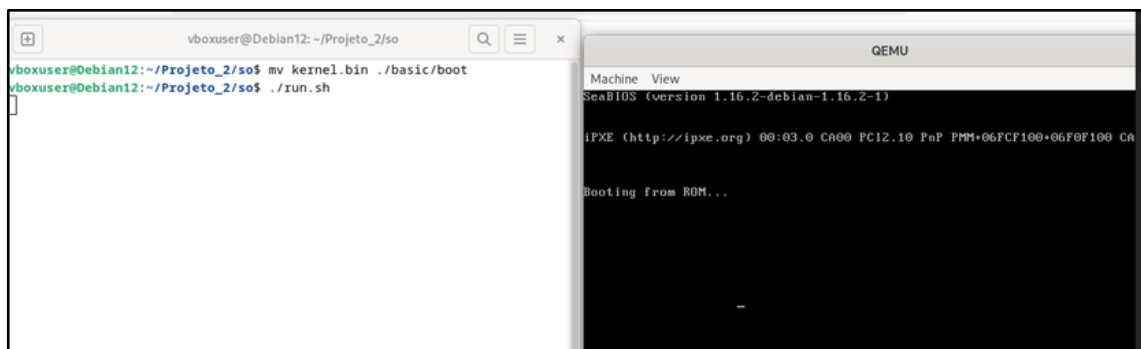


Figura 38 – Testado com sucesso.

Agora vamos gerar a nossa ISO para disponibilizar para testes em outras máquinas utilizando nosso script `make.sh`.

```
vboxuser@Debian12:~/Projeto_2/so$ ./make.sh
xorriso 1.5.4 : RockRidge filesystem manipulator, libburnia project.

Drive current: -outdev 'stdio:basic.iso'
Media current: stdio file, overwriteable
Media status : is blank
Media summary: 0 sessions, 0 data blocks, 0 data, 48.4g free
Added to ISO image: directory '/'='/tmp/grub.3zWXCi'
xorriso : UPDATE :    335 files added in 1 seconds
Added to ISO image: directory '/'='/home/vboxuser/Projeto_2/so/basic'
xorriso : UPDATE :    344 files added in 1 seconds
xorriso : NOTE : Copying to System Area: 512 bytes from file '/usr/lib/grub/i386-pc/boot_hybrid.img'
ISO image produced: 4644 sectors
Written to medium : 4644 sectors at LBA 0
Writing to 'stdio:basic.iso' completed successfully.

vboxuser@Debian12:~/Projeto_2/so$ ls
basic      build.sh  kc.o      kernel.c  make.sh   screen.c
basic.iso  kasm.o   kernel.asm link.ld   run.sh    screen.o
vboxuser@Debian12:~/Projeto_2/so$
```

Figura 39 – ISO gerada com sucesso.

Agora a nossa estrutura de arquivos ficou mais completa conforme a Figura 40 abaixo.

```

vboxuser@Debian12:~$ tree Projeto_2
Projeto_2
├── setup.sh
├── so
│   ├── basic
│   │   ├── boot
│   │   │   ├── grub
│   │   │   │   └── grub.cfg
│   │   │   └── kernel.bin
│   │   └── include
│   │       ├── screen.h
│   │       ├── string.h
│   │       ├── system.h
│   │       └── types.h
│   ├── basic.iso
│   ├── build.sh
│   ├── kasm.o
│   ├── kc.o
│   ├── kernel.asm
│   ├── kernel.c
│   ├── link.ld
│   ├── make.sh
│   ├── run.sh
│   ├── screen.c
│   └── screen.o
6 directories, 18 files

```

Figura 40 – Estrutura de arquivos atualizada.

Enviamos para a nossa máquina os arquivos da máquina guest para a hospedeira utilizando os mesmos meios da tarefa 1.

```

C:\Users\ggust>scp -r vboxuser@192.168.0.107:/home/vboxuser/Projeto_2 C:/
vboxuser@192.168.0.107's password:
string.h                                100% 91    57.4KB/s  00:00
screen.h                               100% 148    93.2KB/s  00:00
types.h                                100% 293   183.3KB/s  00:00
system.h                               100% 134    84.5KB/s  00:00
kernel.bin                             100% 7144   4.4MB/s  00:00
grub.cfg                                100% 103    38.3KB/s  00:00
kasm.o                                  100% 512   241.3KB/s  00:00
kc.o                                    100% 1224   763.2KB/s  00:00
run.sh                                  100% 60     25.5KB/s  00:00
build.sh                                100% 175    83.2KB/s  00:00
link.ld                                 100% 131    61.8KB/s  00:00
screen.c                                100% 3786    2.3MB/s  00:00
kernel.asm                              100% 144    65.7KB/s  00:00
basic.iso                               100% 9288KB 79.8MB/s  00:00
make.sh                                 100% 43    27.2KB/s  00:00
screen.o                                 100% 3768    2.3MB/s  00:00
kernel.c                                 100% 172   107.9KB/s  00:00
setup.sh                                100% 547   342.9KB/s  00:00

C:\Users\ggust>cd Projeto_2
O sistema não pode encontrar o caminho especificado.

C:\Users\ggust>cd C:/Projeto_2

C:\Projeto_2>dir
O volume na unidade C não tem nome.
O Número de Série do Volume é 94C8-E721

Pasta de C:\Projeto_2

10/12/2023  23:41    <DIR>          .
10/12/2023  23:41    <DIR>          ..
10/12/2023  23:41                547 setup.sh
10/12/2023  23:41    <DIR>          so
                        1 arquivo(s)        547 bytes
                        3 pasta(s)    210.900.471.808 bytes disponíveis

C:\Projeto_2>_

```

Figura 41 – Procedimento de transferência de arquivo SCP.

Agora iremos criar nossa máquina virtual com a nossa ISO. Como já havíamos uma imagem antes, bastou excluir a ISO antiga em armazenamento e adicionar um disco óptico novo no lugar.

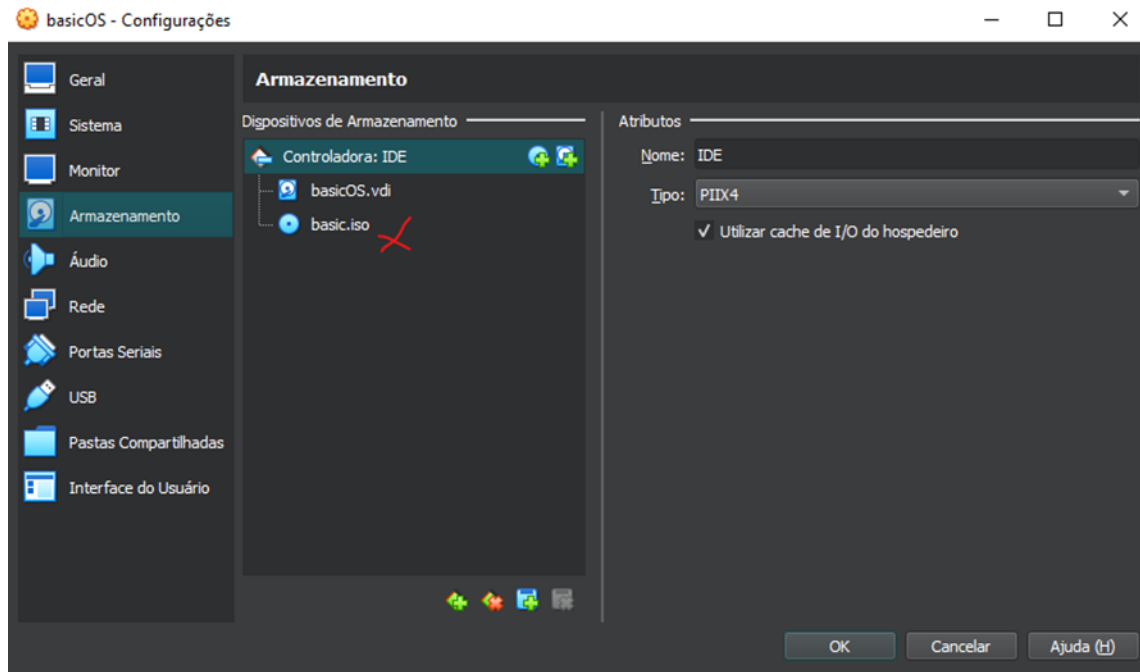


Figura 42 – Excluindo ISO antiga.

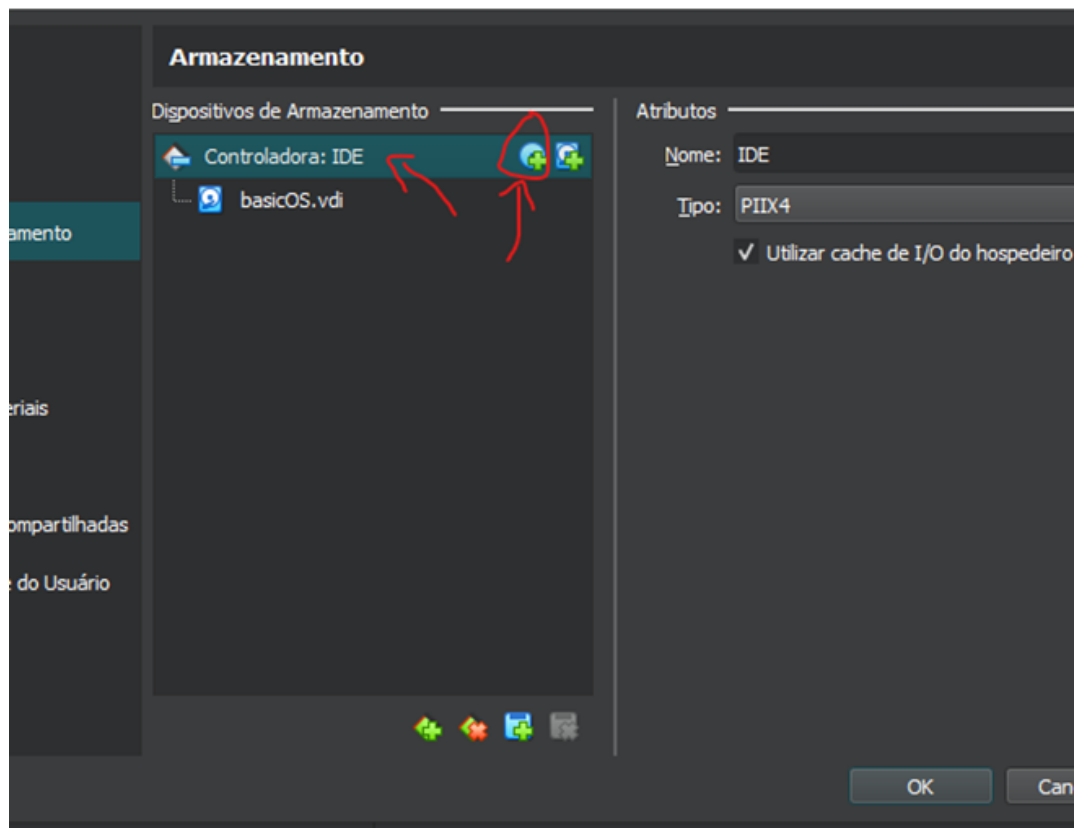


Figura 43 – Adicionando ISO nova.

Clique em acrescentar e selecione a ISO no local que salvamos.

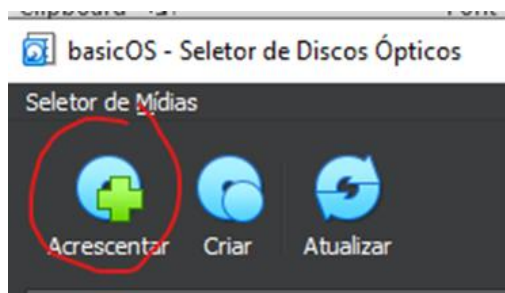


Figura 44 – Acrescentar ISO nova.

Após selecionar, clique em escolher para aplicar.

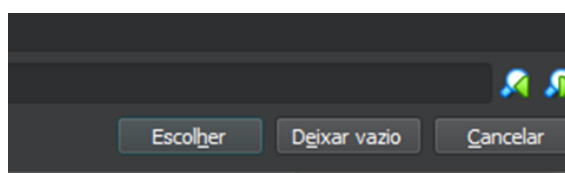


Figura 45 – Aplique a ISO nova.

Pronto, agora concluímos a tarefa 2, onde temos um cursor ponteiro na tela piscando na posição exata que foi solicitada.

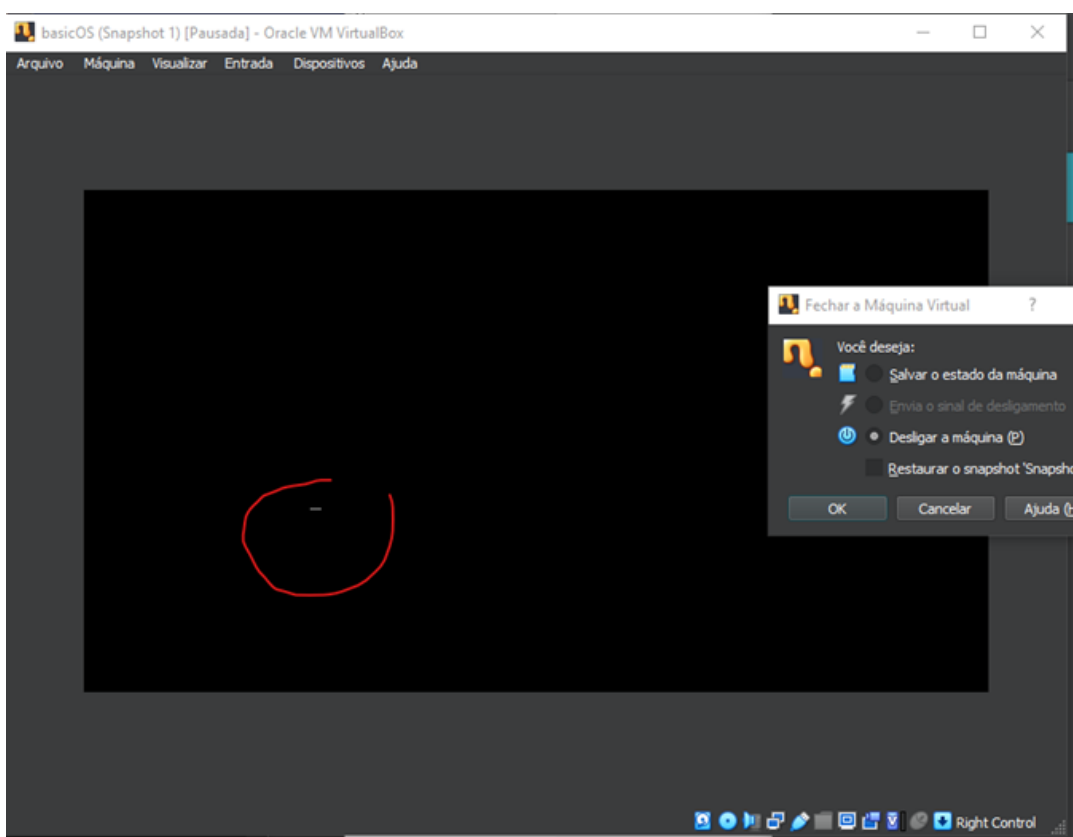


Figura 46 – Resultado final.

3. Considerações finais

Ao trilharmos os primeiros passos no desenvolvimento de um Sistema Operacional (SO), alcançamos conquistas notáveis. A geração bem-sucedida de kernels executáveis em formatos ".bin" e ISO destaca-se como um marco crucial. Essa realização não apenas ampliou nosso entendimento prático, permitindo testes em ambientes virtuais, mas também estabeleceu a base para futuros avanços.

Nossa jornada incluiu uma compreensão profunda da estrutura de boot, desde a fase inicial do carregamento do kernel até a criação de um ambiente operacional funcional. Essa familiaridade essencial nos posicionou para lidar com operações mais avançadas nos estágios subsequentes do desenvolvimento.

Exploramos operações de Input/Output (I/O) em tela em níveis mais baixos, manipulando diretamente o framebuffer de texto na memória de vídeo VGA. Funções como limpar tela, rolar para cima e imprimir caracteres proporcionaram uma interação direta com o ambiente de tela, vital em sistemas operacionais.

Em suma, cumprimos metas fundamentais, construímos uma base sólida de conhecimento e avançamos com êxito nos primeiros estágios do desenvolvimento de um SO. Estas realizações representam um progresso significativo em direção a uma compreensão mais profunda e aquisição de habilidades práticas no universo desafiador do desenvolvimento de sistemas operacionais.

4. Referências

- [1] Debian, (2023) “Instalando o Debian via Internet”, <https://www.debian.org/distrib/netinst>, Junho.
- [2] Nick Clifton, (2022) “The linker’s warnings about executable stacks and segments”, <https://www.redhat.com/en/blog/linkers-warnings-about-executable-stacks-and-segments>, Setembro.
- [3] Dalton Matsuo Tavares, (2025) “Projeto 2 – Primeiros passos para o projeto de um SO”, material disponibilizado no google classroom e SIGAA, Janeiro.
- [4] Sourceware, (2023) “LD”, <https://sourceware.org/binutils/docs/ld/>, Dezembro.
- [5] Robin Miyagi, (2013) “Introduction to GCC Inline Asm”, <https://asm.sourceforge.net/articles/rmiyagi-inline-asm.txt>, Setembro.
- [6] Amine Hajyoussef and Kevin Panko, (2014) “Low level I/O access using outb and inb”, <https://stackoverflow.com/questions/8960620/low-level-i-o-access-using-outb-and-inb>, Junho.